



USTP FINAL PROJECT REPORT

PROJECT: “DESIGN AND UVM BASED VERIFICATION OF AXI TO APB BUS PROTOCOL”

GROUP No. # 08	
MEMBER # 01	ABDUL RAFAY
MEMBER # 02	M. SULEMAN SHAH
MEMBER # 03	BILAL FAISAL
MEMBER # 04	ABDUL NAFAY

INSTRUCTOR NAME: SIR ZIA AHMED SHAH

ACKNOWLEDGEMENT:

We would like to express our sincere gratitude to our project supervisor “Sir Zia Ahmed Shah” for providing us with guidance, technical direction, and valuable feedback throughout the development of this project.

We are also thankful to the Sir Syed University of Engineering & Technology, for providing the academic environment and resources required to complete this work.

We would further like to acknowledge the instructors and trainers involved in Digital IC Design and Verification activities for helping us develop practical knowledge of RTL design, SystemVerilog, UVM, bus protocols, simulation, and functional verification.

Finally, we are grateful to our families, friends, and colleagues for their continuous support and encouragement during the completion of this project.

ABSTRACT:

Modern System-on-Chip (SoC) designs commonly use multiple communication protocols to connect high-performance processing components with lower-speed peripheral devices. AXI4-Lite and APB are two widely used AMBA-based protocols that serve different purposes. AXI4-Lite provides a lightweight memory-mapped interface suitable for control and register-based communication, while APB provides a simple and low-complexity interface for peripheral devices.

This project presents the design and UVM-based verification of an AXI4-Lite to APB bus protocol bridge. The bridge receives AXI4-Lite read and write transactions, processes them through the bridge control logic, transfers requests through asynchronous FIFO-based communication, and converts them into APB transactions. An APB memory model is used as the peripheral-side device for functional verification.

The design was verified using SystemVerilog and Universal Verification Methodology (UVM). The verification environment includes sequence items, sequences, sequencer, driver, monitor, scoreboard, coverage components, and a UVM test. Directed and constrained-random transactions were used to exercise different read/write operations, addresses, data values, and AXI write-channel timing conditions.

The final regression successfully completed 20 AXI4-Lite transactions with zero UVM errors and zero warnings. Functional coverage achieved 100% operation coverage, 87.5% address coverage, and 87.5% operation $\times$ address cross coverage. Read-after-write operations were also successfully checked through the scoreboard, demonstrating correct data transfer between the AXI4-Lite interface and the APB memory.

The project provides practical experience in bus protocol conversion, RTL design, clock-domain communication, SystemVerilog verification, UVM-based verification environments, functional coverage, and waveform-based debugging.

TABLE OF CONTENTS:

1. INTRODUCTION

- 1.1 Purpose
- 1.2 Scope
- 1.3 Project Background
- 1.4 Motivation
- 1.5 Project Objectives
- 1.6 Problem Statement
- 1.7 Organization of the Report

2. LITERATURE REVIEW

3. PROJECT PLANNING AND MANAGEMENT

- 3.1 SWOT Analysis
 - 3.1.1 Strengths
 - 3.1.2 Weaknesses
 - 3.1.3 Opportunities
 - 3.1.4 Threats

4. OVERALL SYSTEM DESCRIPTION

- 4.1 Product Perspective
- 4.2 Product Functions
- 4.3 Operating Environment
- 4.4 Design and Implementation Constraints
- 4.5 Assumptions and Dependencies

5. REQUIREMENTS ANALYSIS

- 5.1 Definitions, Acronyms, and Abbreviations
- 5.2 Functional Requirements
- 5.3 Non-Functional Requirements

6. SYSTEM DESIGN

- 6.1 Design Methodology
- 6.2 Overall Architecture
- 6.3 AXI4-Lite Interface
- 6.4 Request FIFO
- 6.5 Write Data FIFO
- 6.6 Readback/Data Path
- 6.7 APB Master
- 6.8 APB Peripheral/Memory

6.9 Clock-Domain Communication

6.10 Top-Level Data Flow

7. IMPLEMENTATION

7.1 RTL Implementation

7.2 AXI4-Lite Transaction Handling

7.3 APB Transaction Handling

7.4 Bridge Control FSM

7.5 FIFO-Based Data Transfer

7.6 Error and Response Handling

8. VERIFICATION AND VALIDATION

8.1 Verification Methodology

8.2 UVM Testbench Architecture

8.3 Verification Components

8.4 Test Cases

8.5 AXI4-Lite Verification Results

8.6 APB Verification Results

8.7 Functional Coverage Analysis

8.8 Scoreboard Results

8.9 Assertions and Protocol Checking

8.10 Regression Results

9. PROJECT IMPACT AND EVALUATION

9.1 Overview

9.2 Impact on Designers and Verification Engineers

9.3 Practical Applications

9.4 Risk Assessment

9.5 Final Results Table

9.6 Waveform and Output Log Analysis

10. CONCLUSION AND FUTURE WORK

10.1 Conclusion

10.2 Limitations

10.3 Future Work

REFERENCES

APPENDICES

CHAPTER 1

INTRODUCTION:

1.1 Purpose

The purpose of this project is to design and verify an AXI4-Lite to APB bus protocol bridge that enables communication between an AXI4-Lite master and an APB-based peripheral.

The bridge converts AXI4-Lite read and write transactions into corresponding APB transactions while handling the differences between the two protocols. The project also develops a UVM-based verification environment to validate the functional behavior of the bridge.

The project provides practical implementation and verification experience in RTL design, SystemVerilog, AMBA bus protocols, asynchronous data transfer, UVM, functional coverage, scoreboarding, and waveform analysis.

1.2 Scope

The scope of this project includes the design and verification of a basic AXI4-Lite to APB bridge. The major scope areas are:

- AXI4-Lite write transaction handling.
- AXI4-Lite read transaction handling.
- AXI address and data channel handling.
- Conversion of AXI4-Lite requests into APB transactions.
- APB read and write transaction generation.
- APB response handling.
- FIFO-based transfer between interface/control sections.
- Basic clock-domain communication.
- APB memory/peripheral modelling.
- SystemVerilog RTL implementation.
- UVM-based verification.
- Functional coverage collection.
- Scoreboard-based data checking.
- Waveform-based debugging.

The current implementation focuses on a basic AXI4-Lite communication model and does not implement advanced AXI features such as bursts, transaction IDs, out-of-order transactions, or multiple APB slaves.

1.3 Project Background

System-on-Chip designs contain different types of hardware components that communicate using standardized interfaces. A high-performance processor or master may require a flexible communication interface, while simple peripherals such as timers, GPIO controllers, UART registers, and configuration registers usually require a simpler bus.

The Advanced Microcontroller Bus Architecture (AMBA) provides standardized protocols for communication between different components.

AXI4-Lite is commonly used for simple memory-mapped control and register accesses. It contains separate channels for write address, write data, write response, read address, and read data.

APB is designed for simple and low-power peripheral communication. It uses a straightforward transfer mechanism based primarily on PSEL, PENABLE, PWRITE, address, and data signals.

Since AXI4-Lite and APB have different transaction structures, a bridge is required to translate transactions from one protocol to the other.

1.4 Motivation

The motivation for this project is to gain practical understanding of how different AMBA protocols communicate inside an SoC and how protocol conversion can be implemented at RTL level. The project was also motivated by the need to develop practical skills in:

- Digital IC design.
- RTL development.
- SystemVerilog.
- AXI4-Lite protocol.
- APB protocol.
- Clock-domain communication.
- FIFO design.

- UVM verification.
- Functional coverage.
- Scoreboard-based checking.
- Waveform analysis.

Instead of studying the protocols only theoretically, this project provides an opportunity to implement and verify an actual bridge architecture.

1.5 Project Objectives

The main objectives of the project are:

- To understand the architecture and transaction behavior of AXI4-Lite.
- To understand the architecture and transfer phases of APB.
- To design an RTL-based AXI4-Lite to APB bridge.
- To handle AXI4-Lite read and write transactions.
- To generate corresponding APB read and write transactions.
- To use FIFO-based communication for transferring transaction information.
- To develop an APB memory/peripheral model for verification.
- To create a SystemVerilog UVM verification environment.
- To verify different transaction types and address conditions.
- To implement functional coverage.
- To verify read data using a scoreboard.
- To analyze simulation waveforms and output logs.
- To identify potential areas for future improvement.

1.6 Problem Statement

In an SoC, different components may use different communication protocols. A processor or control master using AXI4-Lite cannot directly communicate with an APB peripheral without a suitable interface conversion mechanism. The problem addressed by this project is therefore:

- How can an AXI4-Lite master communicate reliably with an APB-based peripheral through a hardware bridge while maintaining correct address, data, control, and response information?
- The project addresses this problem by designing an AXI4-Lite to APB bridge that receives AXI transactions, processes them internally, generates APB transfers, and returns the APB result to the AXI4-Lite side.

1.7 Organization of the Report

This report is organized into ten major chapters.

- Chapter 1 introduces the project, its purpose, scope, motivation, objectives, and problem statement.
- Chapter 2 discusses the relevant background and literature related to AMBA, AXI4-Lite, APB, bus bridges, and verification.
- Chapter 3 presents project planning and SWOT analysis.
- Chapter 4 describes the overall system and its operating environment.
- Chapter 5 presents the functional and non-functional requirements.
- Chapter 6 explains the system architecture and design methodology.
- Chapter 7 discusses the RTL implementation.
- Chapter 8 presents the verification methodology, test cases, functional coverage, scoreboarding, and regression results.
- Chapter 9 evaluates the final system and discusses its practical impact, results, waveforms, and logs.
- Chapter 10 presents the conclusion, limitations, and future work.

CHAPTER 2

LITERATURE REVIEW:

2.1 AMBA Architecture

The Advanced Microcontroller Bus Architecture (AMBA) is a family of on-chip communication protocols used to connect different components within an SoC.

AMBA protocols provide standardized communication mechanisms between processors, memory controllers, high-performance components, and peripheral devices.

Different AMBA protocols target different performance and complexity requirements.

2.2 AXI4-Lite

AXI4-Lite is a simplified version of the AXI protocol designed primarily for register-based and control-oriented communication.

The AXI4-Lite interface contains five independent channels:

- Write Address Channel
- Write Data Channel
- Write Response Channel
- Read Address Channel
- Read Data Channel

The primary handshake mechanism is based on the VALID and READY signals.

A transfer occurs when both the corresponding VALID and READY signals are asserted.

AXI4-Lite does not support burst transfers, making it suitable for simple control and configuration accesses.

AXI TO APB BUS PROTOCOL

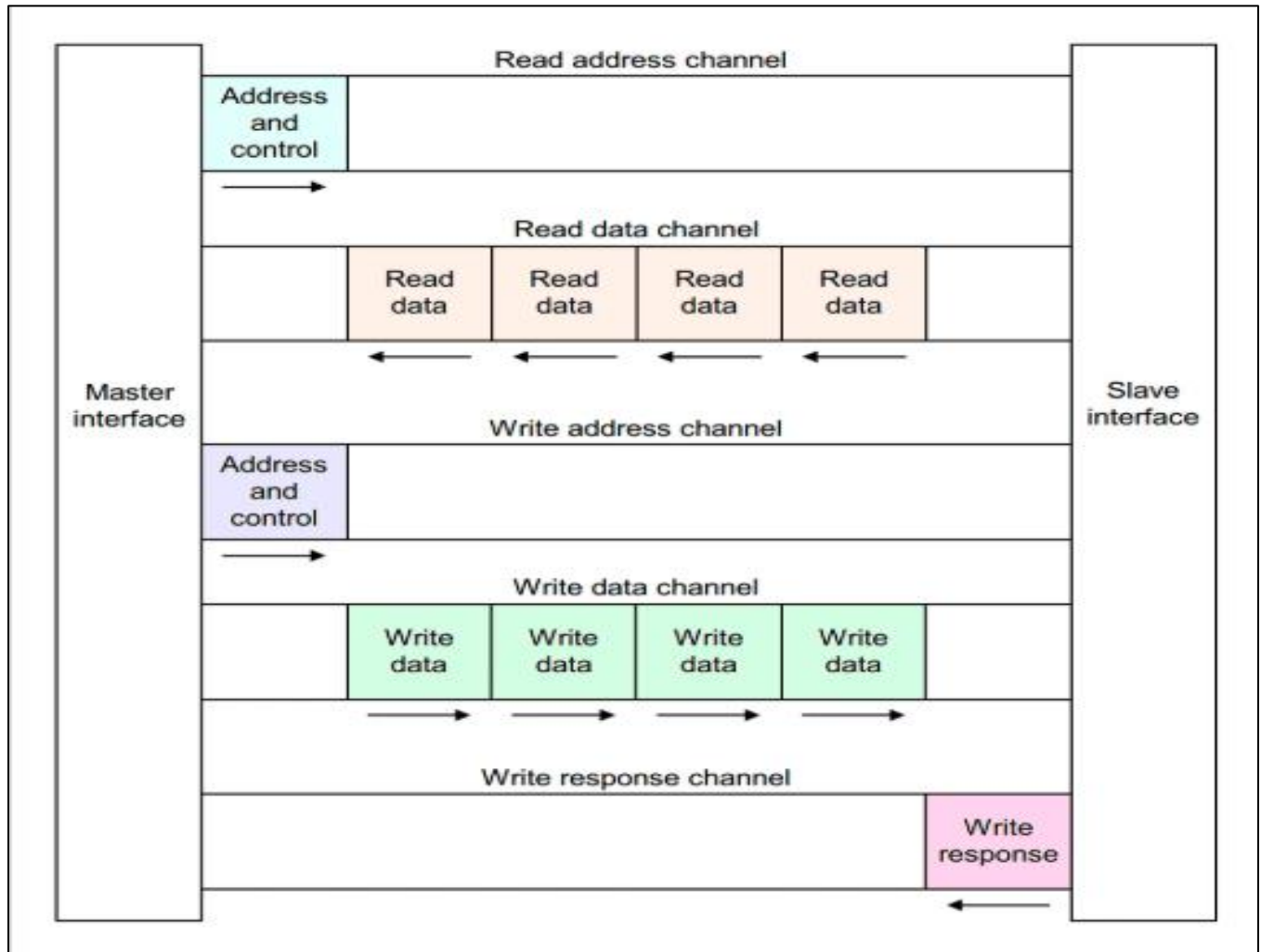


Figure 1: AXI MASTER AND SLAVE CONNECTED VIA THE FIVE AXI CHANNELS

2.3 APB

The Advanced Peripheral Bus is designed for simple peripheral communication.

An APB transfer generally consists of:

- IDLE
- SETUP
- ACCESS

During the setup phase, the peripheral is selected using PSEL. During the access phase, PENABLE is asserted. For write transactions, PWRITE is asserted and PWDATA contains the write data. For read transactions, PWRITE is deasserted and the peripheral provides data through PRDATA. PREADY indicates completion of the APB transfer, while PSLVERR indicates an APB error condition.

Advanced Peripheral Bus (APB)

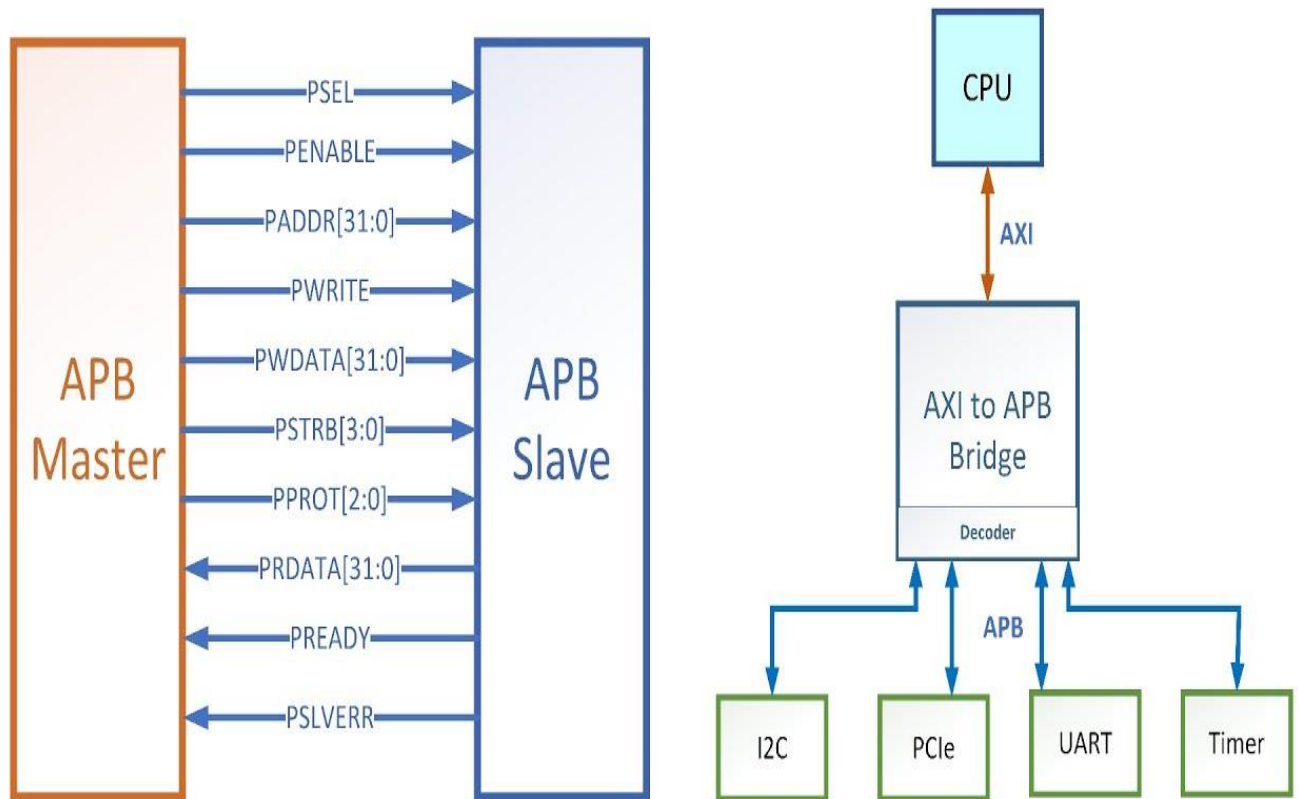


Figure 2: ADVANCED PERIPHERAL BUS INTERFACE SIGNALS

2.4 AXI4-Lite to APB Bridge

An AXI4-Lite to APB bridge provides an interface between the two protocols. The bridge must:

- Receive AXI4-Lite requests.
- Handle separate AXI write address and write data channels.
- Store transaction information when necessary.
- Generate APB control signals.
- Wait for the APB response.
- Return the result to the AXI4-Lite master.

The bridge therefore acts as a protocol conversion layer.

AXI TO APB BUS PROTOCOL

AXI to APB Bridge – Subblocks Specification

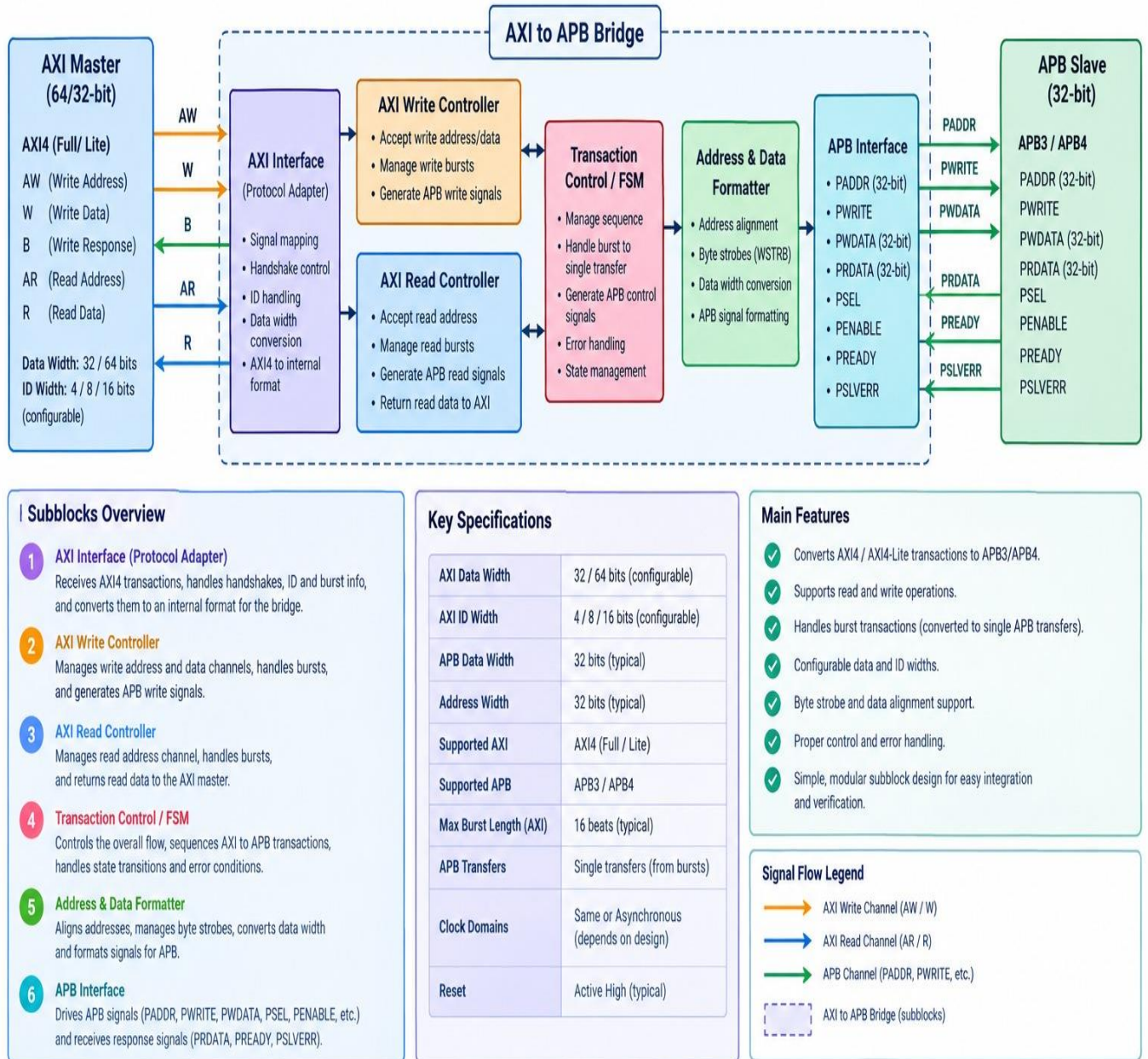


Figure 1: AXI TO APB BRIDGESUBBLOCKS SPECIFICATION

2.5 UVM-Based Verification

Universal Verification Methodology provides a reusable framework for verifying SystemVerilog designs.

A typical UVM environment consists of:

- Sequence item
- Sequence
- Sequencer
- Driver
- Monitor
- Agent
- Scoreboard
- Coverage
- Environment
- Test

For this project, UVM is used to generate AXI4-Lite transactions and observe the resulting behaviour of the bridge.

2.6 Summary of Literature Review

The reviewed concepts demonstrate that AXI4-Lite and APB have different architectures and transaction mechanisms. Therefore, protocol conversion requires dedicated control logic.

The project combines these concepts with FIFO-based transfer and UVM verification to create and evaluate a complete protocol bridge.

CHAPTER 3

PROJECT PLANNING AND MANAGEMENT:

3.1 SWOT Analysis

A SWOT analysis was performed to identify the strengths, weaknesses, opportunities, and threats associated with the project.

3.1.1 Strengths

- Uses standardized AMBA-based protocols.
- Provides practical RTL design experience.
- Supports AXI4-Lite read and write transactions.
- Uses FIFO-based transaction transfer.
- Includes a UVM verification environment.
- Includes functional coverage.
- Uses a scoreboard for data verification.
- Supports waveform-based debugging.
- Provides a reusable foundation for future improvements.

3.1.2 Weaknesses

- Current implementation focuses on AXI4-Lite rather than full AXI4.
- Burst transactions are not supported.
- Multiple outstanding transactions are not targeted.
- The current APB side uses a basic memory/peripheral model.
- Error scenarios are limited compared with a production-level verification environment.
- Coverage can be improved further by adding additional address and protocol scenarios.

3.1.3 Opportunities

The project can be extended to:

- Support additional APB peripherals.
- Support more extensive AXI protocol scenarios.
- Add stronger assertion-based verification.
- Increase functional coverage.
- Add protocol compliance checking.
- Perform synthesis and timing analysis.

- Evaluate area and power.
- Extend the design toward reusable IP development.

3.1.4 Threats

Potential project risks include:

- Incorrect protocol timing.
- Clock-domain synchronization issues.
- FIFO overflow or underflow.
- Incorrect handling of separate AXI address and data channels.
- Incorrect APB setup/access sequencing.
- Insufficient verification coverage.
- Tool-specific simulation behavior.

These risks are reduced through simulation, scoreboard checking, functional coverage, assertions, and waveform analysis.

CHAPTER 4

OVERALL SYSTEM DESCRIPTION:

4.1 Product Perspective

The project is designed as an interface bridge between an AXI4-Lite master and an APB peripheral. The high-level architecture can be represented as:

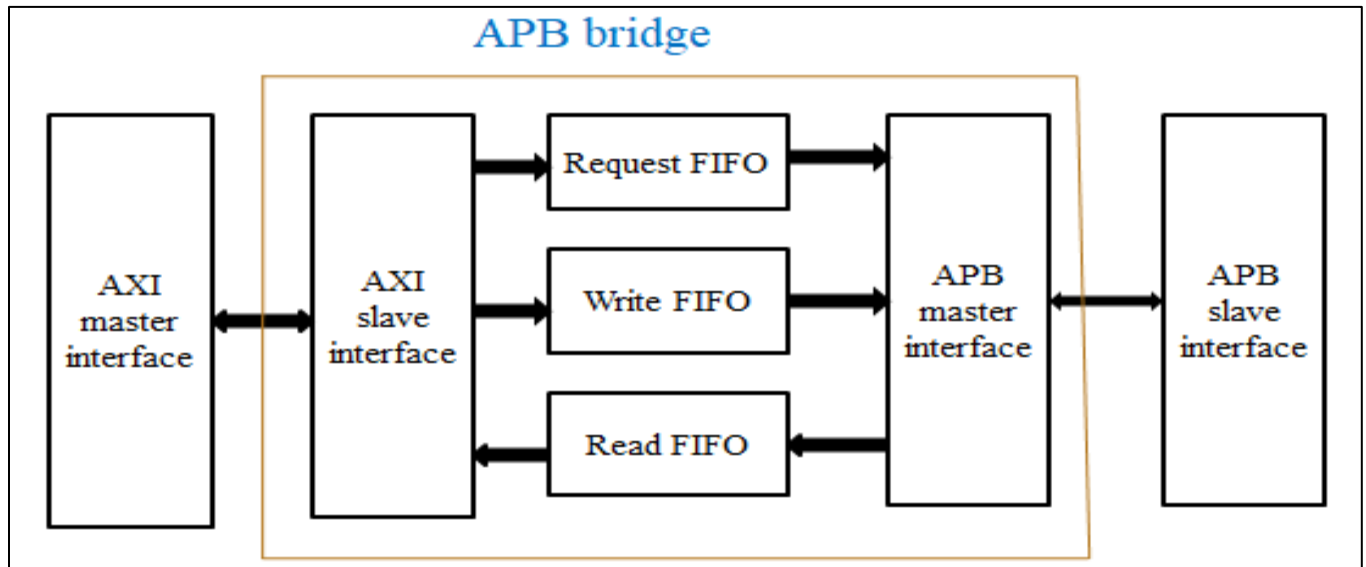


Figure 4: BASIC ARCHTECTURE OF AXI TO APB BRIDGE

The APB response is then transferred back through the bridge and returned to the AXI4-Lite master.

4.2 Product Functions

The major functions of the system are:

- Accept AXI4-Lite read requests.
- Accept AXI4-Lite write requests.
- Handle AXI write address and write data independently.
- Store transaction information.
- Generate APB requests.
- Generate APB setup and access phases.
- Transfer address and write data.
- Capture APB read data.

- Capture APB response/error information.
- Generate AXI read responses.
- Generate AXI write responses.

4.3 Operating Environment

The project was developed and verified using:

Component	Environment
HDL	SystemVerilog
Verification	UVM
Simulator	Cadence Xcelium 25.03
Waveform	EPWave/VCD
Coverage	Xcelium Coverage
Testbench	UVM-based
Protocol	AXI4-Lite and APB
DUT	RTL bridge
Peripheral	APB memory model

4.4 Design and Implementation Constraints

The project has the following constraints:

- AXI4-Lite functionality is targeted instead of full AXI4.
- Burst transfers are not implemented.
- Transaction IDs are not used.
- The APB side is based on a simple peripheral/memory model.
- The design is intended primarily for functional learning and verification.
- Advanced SoC-level optimization is outside the current scope.

4.5 Assumptions and Dependencies

The following assumptions are made:

- AXI4-Lite transactions contain valid addresses and control information.
- The APB peripheral responds according to the expected APB transfer behavior.
- FIFO communication correctly transfers transaction information.
- Clock and reset signals are available to the bridge.
- The testbench provides valid stimulus.
- The APB memory model provides deterministic read/write behavior.

CHAPTER 5

REQUIREMENTS ANALYSIS:

5.1 Definitions, Acronyms, and Abbreviations

Term	Meaning
AXI	Advanced eXtensible Interface
AXI4-Lite	Lightweight AXI memory-mapped protocol
APB	Advanced Peripheral Bus
AMBA	Advanced Microcontroller Bus Architecture
RTL	Register Transfer Level
UVM	Universal Verification Methodology
DUT	Design Under Test
FSM	Finite State Machine
FIFO	First-In First-Out
ACLK	AXI clock
PCLK	APB clock
SVA	SystemVerilog Assertions
UVC	UVM Verification Component
Functional Coverage	Measurement of exercised functional scenarios

5.2 Functional Requirements

- **FR-01: AXI Write Support**
The bridge shall accept AXI4-Lite write address and write data information.
- **FR-02: AXI Read Support**
The bridge shall accept AXI4-Lite read address requests.
- **FR-03: APB Write Generation**
The bridge shall convert AXI write transactions into APB write transfers.
- **FR-04: APB Read Generation**
The bridge shall convert AXI read transactions into APB read transfers.
- **FR-05: Address Transfer**
The bridge shall transfer the AXI address to the APB address interface.
- **FR-06: Write Data Transfer**
The bridge shall transfer AXI write data to APB PWDATA.

- **FR-07: Read Data Transfer**
The bridge shall capture APB PRDATA and return it through the AXI read response.
- **FR-08: Response Handling**
The bridge shall generate an appropriate AXI response after APB transaction completion.
- **FR-09: FIFO Communication**
The bridge shall use FIFO-based storage/transfer where required for transaction communication.
- **FR-10: Verification**
The implementation shall be verified using a SystemVerilog/UVM-based environment.

5.3 Non-Functional Requirements

The system should:

- Be modular.
- Be understandable and maintainable.
- Follow RTL design principles.
- Support simulation-based verification.
- Provide observable waveforms.
- Provide functional coverage.
- Provide scoreboard-based checking.
- Allow future extension.
- Avoid unnecessary design complexity.

CHAPTER 6

SYSTEM DESIGN:

6.1 Design Methodology

The project follows a modular RTL design methodology. The design is divided into multiple functional blocks:

- AXI4-Lite interface.
- Request storage.
- Write-data storage.
- APB control logic.
- APB peripheral.
- Response/data transfer logic.

Each block has a specific responsibility, making the overall design easier to understand, debug, and verify.

6.2 Overall Architecture

The overall architecture consists of:

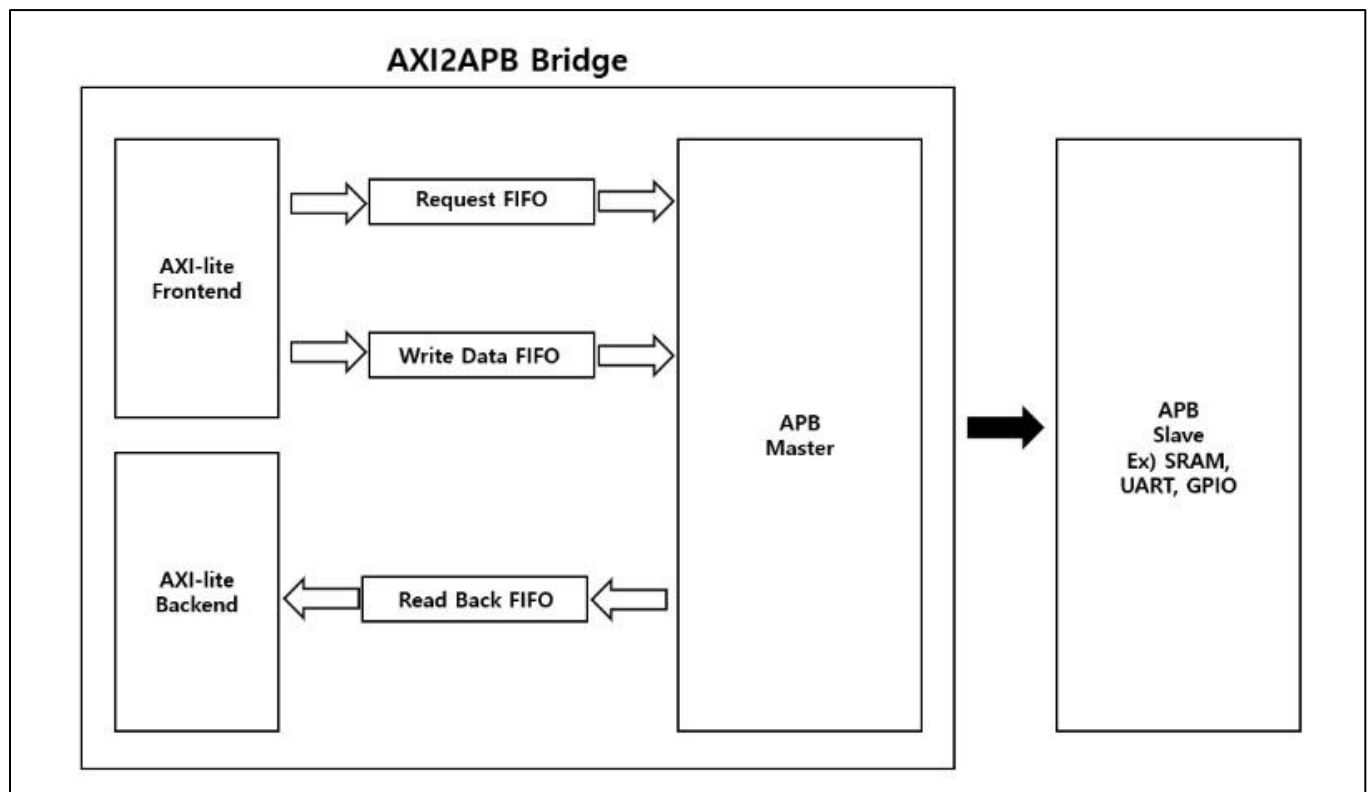


Figure 5: OVERALL BLOCK DIAGRAM AXI TO APB

6.3 AXI4-Lite Interface

The AXI4-Lite interface uses five channels:

- **Write Address Channel**

Signals include:

- ❖ AWADDR
- ❖ AWVALID
- ❖ AWREADY

- **Write Data Channel**

Signals include:

- ❖ WDATA
- ❖WSTRB
- ❖WVALID
- ❖WREADY

- **Write Response Channel**

Signals include:

- ❖ BRESP
- ❖BVALID
- ❖BREADY

- **Read Address Channel**

Signals include:

- ❖ ARADDR
- ❖ ARVALID
- ❖ ARREADY

- **Read Data Channel**

Signals include:

- ❖ RDATA
- ❖ RRESP
- ❖ RVALID
- ❖ RREADY

The bridge monitors these channels and collects the information required to construct APB transactions.

AXI TO APB BUS PROTOCOL

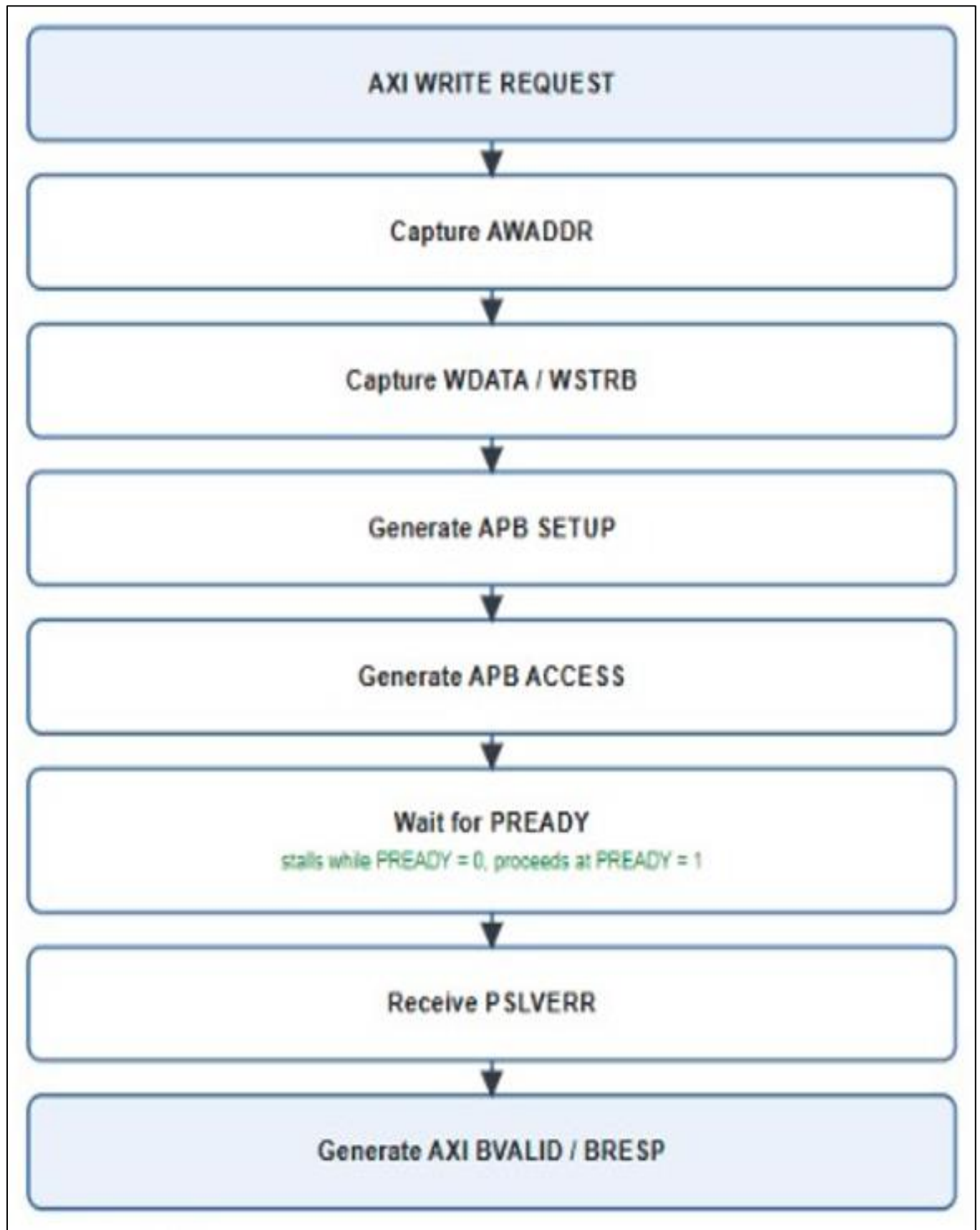


Figure 2: AXI WRITE TRANSACTION FLOW

AXI TO APB BUS PROTOCOL

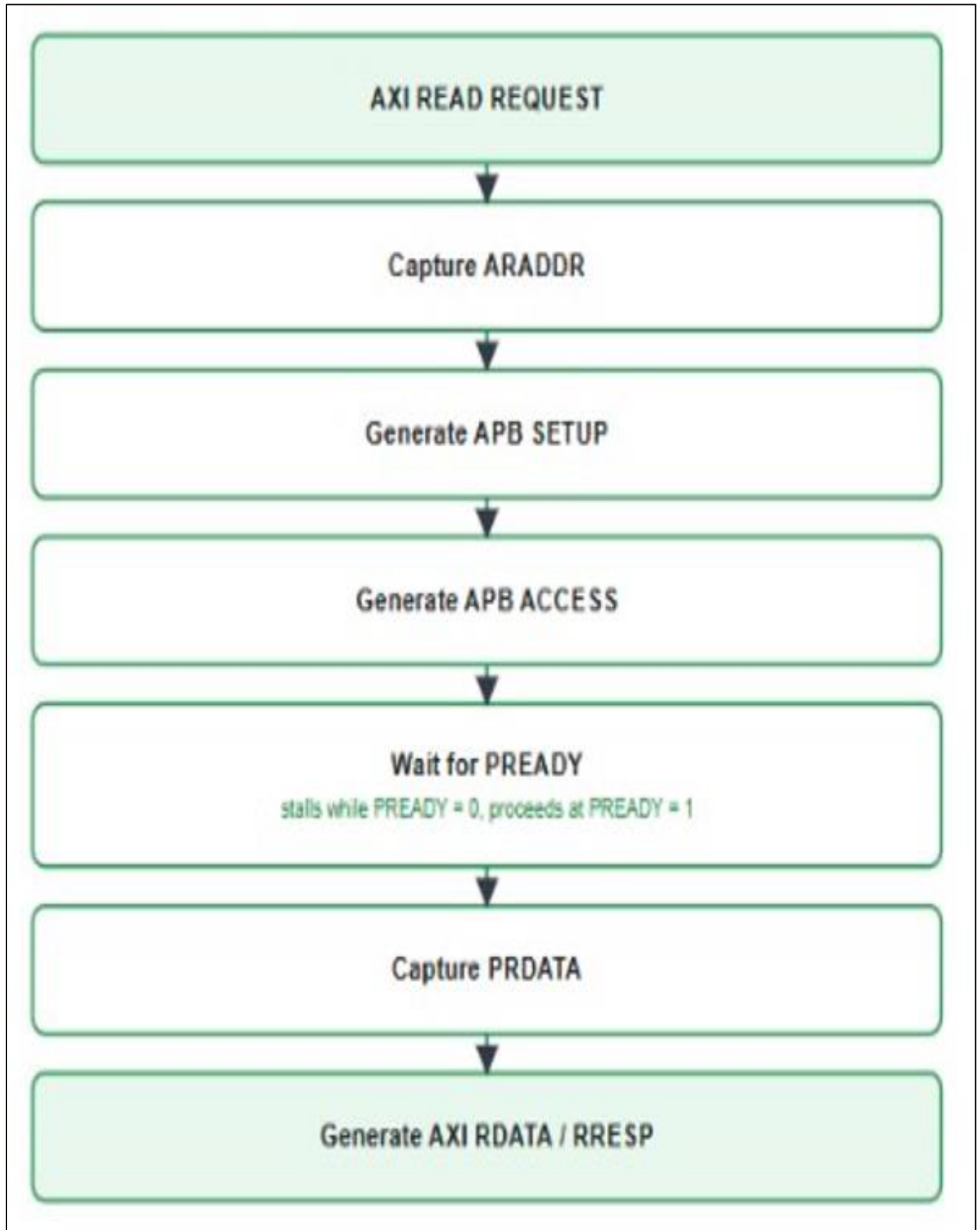


Figure 3: AXI READ TRANSACTION FLOW

6.4 Request FIFO

The request FIFO stores transaction control information before it is processed by the APB side. Typical information includes:

- Address.
- Read/write indication.
- Request validity.

The FIFO allows the bridge control logic to process requests in an organized manner.

6.5 Write Data FIFO

Because AXI4-Lite write address and write data channels are independent, the address and data may not arrive at exactly the same time. The write-data FIFO/storage mechanism therefore allows write data to be held until the corresponding write transaction can be processed. The simulation log demonstrated this behavior for separate AXI address/data cases:

BRIDGE CONTROL: Request received. WRITE=1 ADDR=00000080 READY=0

BRIDGE CONTROL: Write data received = d8b356a6

BRIDGE CONTROL: Request received. WRITE=1 ADDR=00000080 READY=1

This demonstrates that the bridge can wait for the required write information before starting the APB transaction.

6.6 Readback/Data Path

For an APB read, the peripheral provides data through PRDATA. The bridge captures this data and returns it through the AXI read-data channel. For example, the verification log showed:

APB MEMORY: READ ADDR=00000010 INDEX=4 DATA=88b687a8

BRIDGE CONTROL: APB RESPONSE received. RDATA=88b687a8 ERROR=0

The corresponding AXI monitor observed:

READ RESPONSE: DATA=88b687a8 RRESP=00:

6.7 APB Master

The APB master is responsible for generating the APB transfer. Important signals include:

- PSEL
- PENABLE
- PWRITE
- PADDR
- PWDATA
- PRDATA
- PREADY
- PSLVERR

The APB master follows the basic setup and access sequence.

6.8 APB Peripheral / Memory

A simple APB memory model was used as the peripheral for verification. The memory model supports:

- APB writes.
- APB reads.
- Address decoding.
- Data storage.
- Data retrieval.

For example:

APB MEMORY: WRITE ADDR=00000020 DATA=c105a040

Later, a read from an address that had previously been written produced:

APB MEMORY: READ ADDR=00000020 INDEX=8 DATA=8cdb109d

The scoreboard compared the returned value with the expected value.

6.9 Clock-Domain Communication

The design includes FIFO-based communication between portions of the bridge that operate around different clock/control domains. The use of asynchronous FIFO structures helps safely transfer transaction information across clock boundaries. This is particularly

AXI TO APB BUS PROTOCOL

important in practical SoC designs where different bus interfaces may operate at different clock frequencies.

6.10 Top-Level Data Flow

The complete transaction flow can be summarized as:

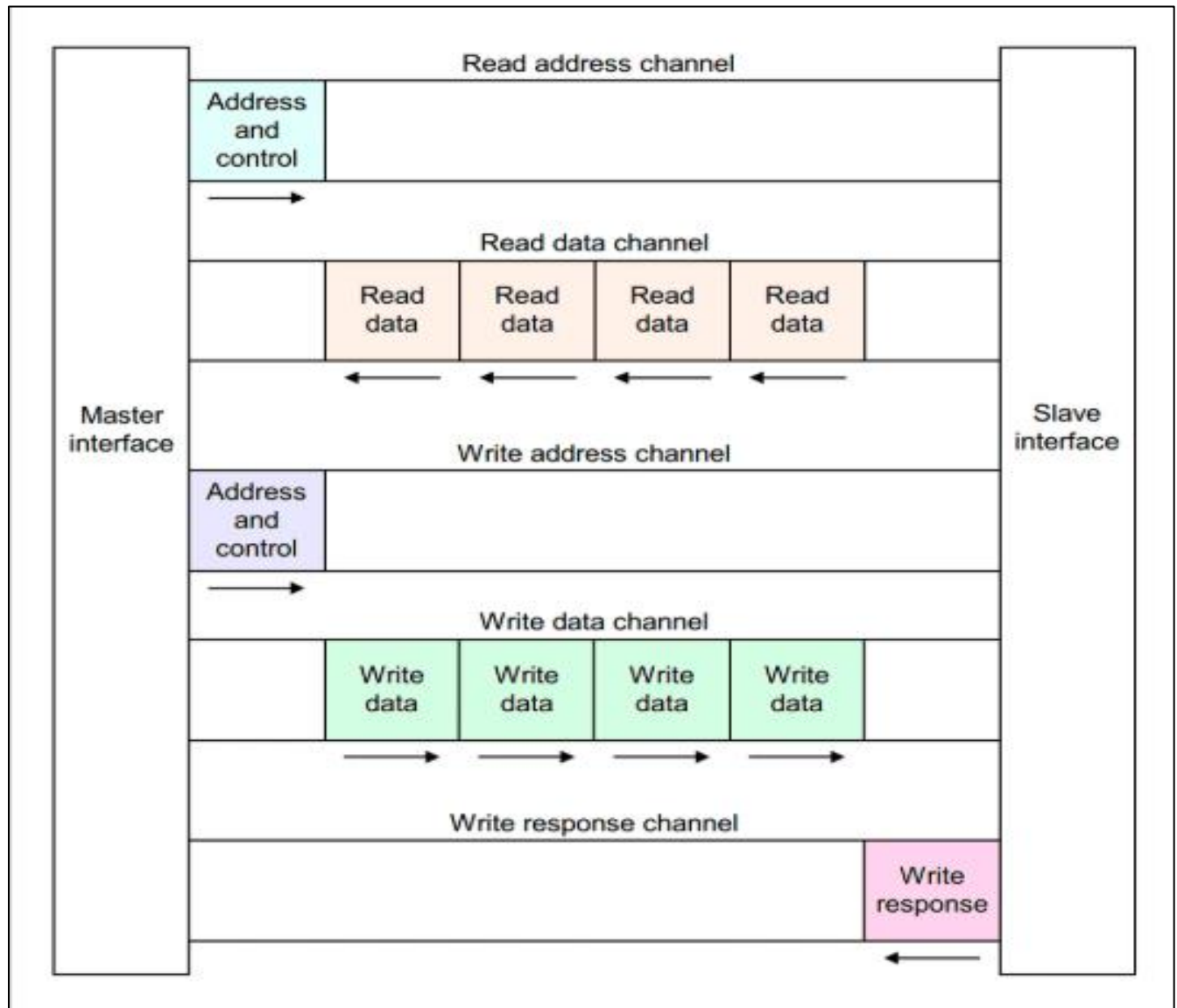


Figure 8: AXI MASTER AND SLAVE CONNECTED VIA THE FIVE AXI CHANNELS

CHAPTER 7

IMPLEMENTATION:

7.1 RTL Implementation

The bridge was implemented using SystemVerilog RTL. The implementation is divided into functional modules to simplify development and verification. The main implementation responsibilities include:

- AXI transaction reception.
- Request storage.
- Write data storage.
- APB transaction generation.
- APB response capture.
- AXI response generation.
- FIFO-based transaction transfer.

7.2 AXI4-Lite Transaction Handling

- **AXI Write**

An AXI write transaction requires both the write address and write data. The bridge waits until the required information is available. The simulation demonstrated both normal and separate address/data cases.

For example:

SEPARATE_AW_W=1

was exercised during the regression.

- **AXI Read**

For an AXI read:

- ❖ The master provides ARADDR.
- ❖ The bridge accepts the address.
- ❖ The request is transferred to the APB side.
- ❖ The APB peripheral returns PRDATA.
- ❖ The bridge provides the data through RDATA.
- ❖ RRESP indicates the AXI response status.

7.3 APB Transaction Handling

The APB master generates the required control signals according to the transaction type.

For writes:

- Address is placed on PADDR.
- Data is placed on PWDATA.
- PWRITE is asserted.
- The APB peripheral is selected.
- The bridge waits for completion.

For reads:

- Address is placed on PADDR.
- PWRITE is deasserted.
- The peripheral returns PRDATA.
- The bridge captures the response.

7.4 Bridge Control FSM

The bridge control logic coordinates the movement of transactions between AXI4-Lite and APB. A simplified conceptual flow is:

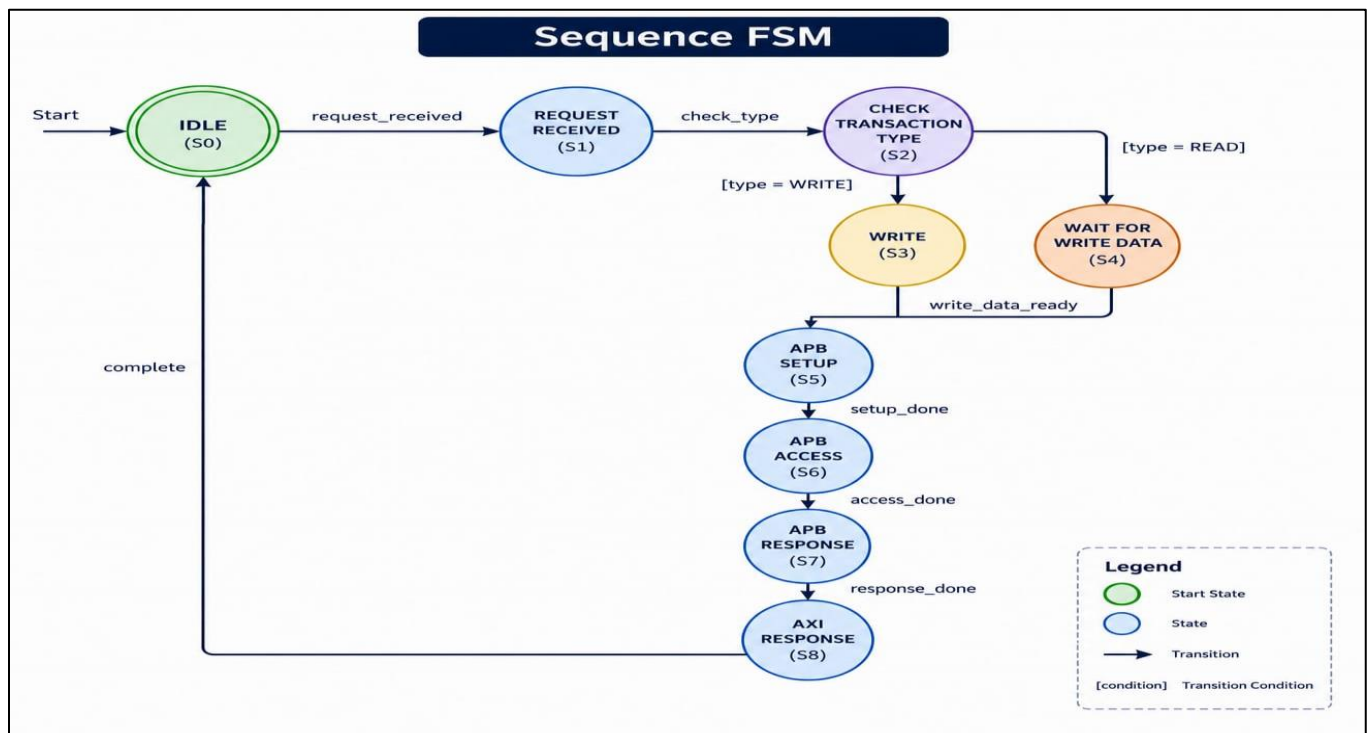


Figure 9: SEQUENCE DIAGRAM GENERAL

AXI TO APB BUS PROTOCOL

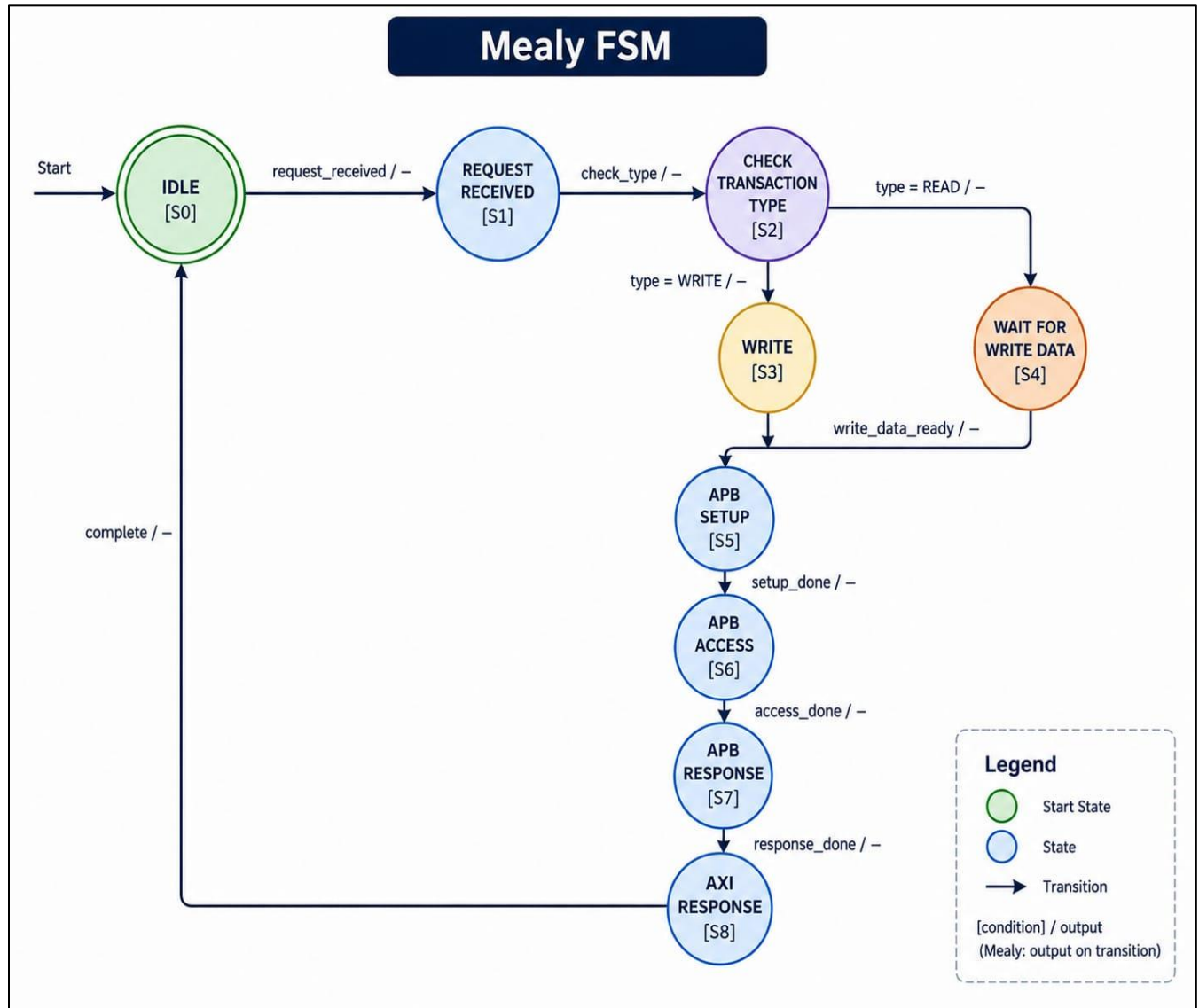


Figure 10: FSM MEALY DIAGRAM

AXI TO APB BUS PROTOCOL

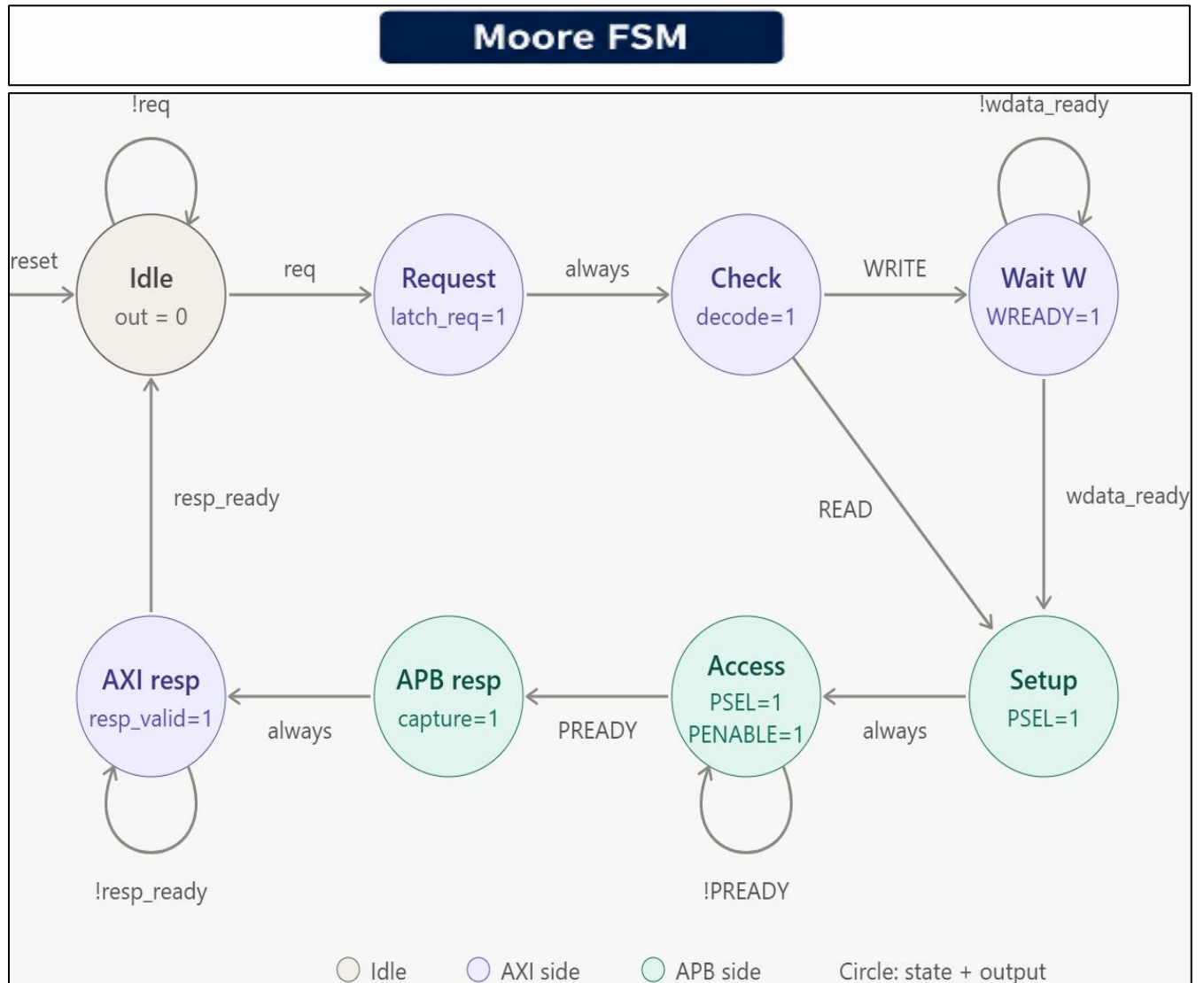


Figure 11: FSM MOORE DIAGRAM

7.5 FIFO-Based Data Transfer

FIFO structures are used to store and transfer transaction information. The use of FIFO-based communication provides:

- Temporary transaction storage.
- Decoupling between processing stages.
- Support for clock-domain communication.
- Better organization of transaction flow.

7.6 Error and Response Handling

The APB side provides PSLVERR for reporting APB transfer errors. The bridge captures the APB response and converts it into the appropriate AXI response behaviour. During the final regression, the tested transactions completed with:

BRESP=00

RRESP=00

ERROR=0

indicating successful transactions for the tested scenarios.

CHAPTER 8

VERIFICATION AND VALIDATION:

8.1 Verification Methodology

The bridge was verified using a SystemVerilog-based UVM environment. The verification methodology follows the following flow:

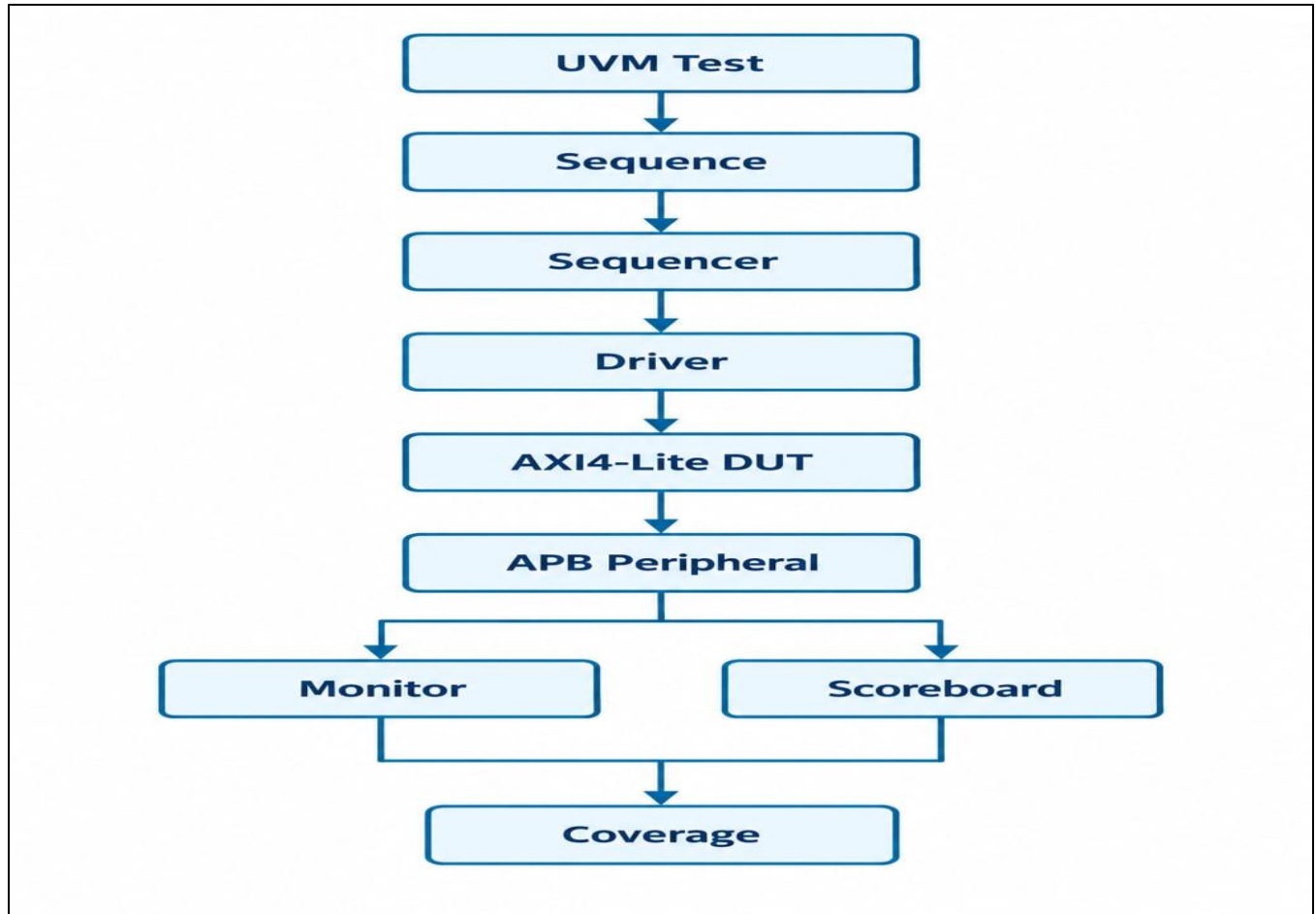


Figure 10: THE FLOW OF VERIFICATION METHODOLOGY

8.2 UVM Testbench Architecture

The main UVM components are:

- **Sequence Item:** Represents an AXI transaction containing fields such as:
 - ❖ Read/write operation.
 - ❖ Address.
 - ❖ Data.
 - ❖ Separate address/data control.

AXI TO APB BUS PROTOCOL

- **Sequence:** Generates AXI transactions for the DUT.
- **Sequencer:** Provides sequence items to the driver.
- **Driver:** Converts sequence items into AXI signal-level activity.
- **Monitor:** Observes AXI transactions from the DUT interface.
- **Scoreboard:** Checks expected and actual transaction results.
- **Coverage:** Measures functional scenarios exercised during simulation.
- **Environment:** Contains the agent, scoreboard, and coverage components.
- **Test:** Controls the overall verification sequence and reports final results.

8.3 Verification Components

The verification environment contains the following major components:

Component	Purpose
AXI Sequence Item	Stores transaction information
AXI Sequence	Generates stimulus
AXI Sequencer	Sends transactions to driver
AXI Driver	Drives AXI interface
AXI Monitor	Observes AXI activity
Scoreboard	Compares expected and actual results
Coverage	Measures functional scenarios
UVM Test	Controls regression

8.4 Test Cases

The final regression exercised different transaction types.

Test Case	Operation	Address Example	Result
TC-01	Read	0x80	PASS
TC-02	Write	0x20	PASS
TC-03	Write	0x30	PASS
TC-04	Read	0x10	PASS
TC-05	Read	0x50	PASS
TC-06	Separate AW/W Write	0x80	PASS
TC-07	Write	0x10	PASS
TC-08	Separate AW/W Write	0x70	PASS
TC-09	Read-after-write	0x10	PASS

AXI TO APB BUS PROTOCOL

TC-10	Read	0x50	PASS
TC-11	Write	0x50	PASS
TC-12	Separate AW/W Write	0x80	PASS
TC-13	Separate AW/W Write	0x40	PASS
TC-14	Separate AW/W Write	0x20	PASS
TC-15	Write	0x70	PASS
TC-16	Read-after-write	0x30	PASS
TC-17	Read-after-write	0x20	PASS
TC-18	Write	0x80	PASS
TC-19	Read-after-write	0x80	PASS
TC-20	Write	0x50	PASS

8.5 AXI4-Lite Verification Results

The final regression successfully completed 20 AXI4-Lite transactions. The UVM test reported:

All 20 AXI transactions completed successfully.

Driver count = 20

The test included both read and write transactions. The regression also exercised separate AXI write address and data behaviour using:

SEPARATE_AW_W=1

The driver, monitor, and scoreboard successfully observed the corresponding transaction activity. The final UVM report showed:

UVM_INFO: 207

UVM_WARNING: 0

UVM_ERROR: 0

UVM_FATAL: 0

Therefore, the final regression completed without UVM errors or fatal failures.

8.6 APB Verification Results

The APB master behavior was verified through the bridge's generated APB transactions and the APB memory responses. The following APB functions were exercised:

APB Signal	Verification Observation
PSEL	Peripheral selected during APB transfer
PENABLE	Used during APB access phase
PWRITE	Differentiated read and write operations
PADDR	Correct AXI address propagated to APB
PWDATA	Write data transferred to APB memory
PRDATA	Read data returned from APB memory
PREADY	Used to indicate APB transaction completion
PSLVERR	Error status observed as no error in tested cases

- **Example APB write:**

BRIDGE CONTROL: Starting APB WRITE, ADDR=00000020 DATA=c105a040

APB MEMORY: WRITE ADDR=00000020 DATA=c105a040

BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0

- **Example APB read:**

BRIDGE CONTROL: Starting APB READ, ADDR=00000010

APB MEMORY: READ ADDR=00000010 INDEX=4 DATA=88b687a8

BRIDGE CONTROL: APB RESPONSE received. RDATA=88b687a8 ERROR=0

These results demonstrate that the bridge successfully generated APB read/write operations and correctly received APB responses.

8.7 Functional Coverage Analysis

Functional coverage was implemented using UVM covergroups. Three major coverage categories were measured:

- Operation coverage.
- Address coverage.
- Operation × address cross coverage.

The final regression produced:

Coverage Type	Result
Operation Coverage	100.00%

AXI TO APB BUS PROTOCOL

Address Coverage	87.50%
Operation × Address Coverage	87.50%

- **Operation Coverage**

Operation coverage reached 100%, meaning both required operation types, read and write, were exercised.

- **Address Coverage**

Address coverage reached 87.50%. This indicates that most of the address bins defined in the coverage model were exercised, while some address scenarios remained unhit.

- **Cross Coverage**

Operation × address coverage reached 87.50%, showing that most combinations between transaction type and address category were exercised.

The remaining coverage can be improved in future regression runs by intentionally generating transactions for the missing address bins and operation/address combinations.

8.8 Scoreboard Results

The scoreboard was used to compare expected and actual transaction results. For write transactions, the scoreboard confirmed that the write address and data were correctly received. For read transactions, the scoreboard maintained previously written values and compared them with the returned read data.

Example:

READ CHECK PASS:

ADDR=00000010

EXPECTED=88b687a8

ACTUAL=88b687a8

Another successful read-after-write result was:

READ CHECK PASS:

ADDR=00000030

EXPECTED=73fa2f86

ACTUAL=73fa2f86

These checks provide evidence that the bridge successfully transferred data between AXI4-Lite and the APB memory.

8.9 Assertions and Protocol Checking

SystemVerilog assertions can be used to verify protocol-level conditions such as:

- Valid/ready handshake behaviour.
- Correct response generation.
- FIFO overflow/underflow conditions.
- APB setup/access sequencing.
- Stable address/data during required transfer phases.
- Reset behaviour.

Assertions provide an additional verification layer beyond scoreboard checking and functional coverage. The project can be extended with a larger assertion set as part of future verification improvements.

8.10 Regression Results

The final UVM regression was executed using Cadence Xcelium 25.03. The simulation completed at:

4070 ns

Final test result:

All 20 AXI transactions completed successfully.

Driver count = 20

Final UVM report:

UVM_INFO: 207

UVM_WARNING: 0

UVM_ERROR: 0

UVM_FATAL: 0

The regression therefore completed successfully without UVM errors or fatal failures.

CHAPTER 9

PROJECT IMPACT AND EVALUATION:

9.1 Overview

The project provides a practical implementation of a protocol bridge and demonstrates how different AMBA interfaces can be integrated through dedicated RTL control logic. It also demonstrates the importance of verification in digital IC design. Instead of relying only on simulation completion, the project uses:

- UVM stimulus generation.
- Monitoring.
- Scoreboarding.
- Functional coverage.
- Waveform analysis.
- Output log analysis.

This provides a more structured verification process:

9.2 Impact on Designers and Verification Engineers

The project provides practical learning in:

- RTL architecture.
- AXI4-Lite transactions.
- APB transactions.
- FIFO-based communication.
- Clock-domain considerations.
- SystemVerilog.
- UVM.
- Functional coverage.
- Scoreboarding.
- Debugging.

The architecture can also serve as a starting point for developing more advanced bridge IP.

9.3 Practical Applications

An AXI4-Lite to APB bridge can be used in SoC architectures where a higher-level AXI-based component needs to access simple APB peripherals. Possible applications include:

- Configuration registers.
- GPIO controllers.
- UART peripherals.
- Timer peripherals.
- Watchdog controllers.
- Low-speed control devices.
- Peripheral register banks.
- Sensor/control interfaces.

For example, a processor connected to an AXI interconnect can access an APB-based UART through an AXI-to-APB bridge

9.4 Risk Assessment

Risk	Possible Effect	Mitigation
Incorrect AXI handshake	Transaction failure	UVM monitoring and checking
Separate AW/W timing	Incorrect write transaction	Dedicated write-data handling
FIFO issue	Lost/corrupted transaction	FIFO verification
APB sequencing error	Invalid peripheral access	APB protocol checking
Incorrect read data	Functional failure	Scoreboard
Insufficient coverage	Unverified scenarios	Functional coverage
Clock-domain issue	Data corruption	FIFO-based synchronization
Reset issue	Unknown initial state	Reset testing

9.5 Final Results Table

The final results of the project are summarized below.

Parameter	Final Result
Simulation Tool	Cadence Xcelium 25.03
Verification Methodology	UVM
Total AXI Transactions	20
Completed Transactions	20

Transaction Completion	100%
UVM Errors	0
UVM Fatal Errors	0
UVM Warnings	0
Operation Coverage	100.00%
Address Coverage	87.50%
Operation × Address Coverage	87.50%
Read/Write Verification	PASS
Read-after-Write Checks	PASS
APB Error in Tested Cases	0
Final Regression	PASS

9.6 Waveform and Output Log Analysis

The simulation waveform provides visual evidence of the transaction flow through the bridge. The waveform includes signals such as:

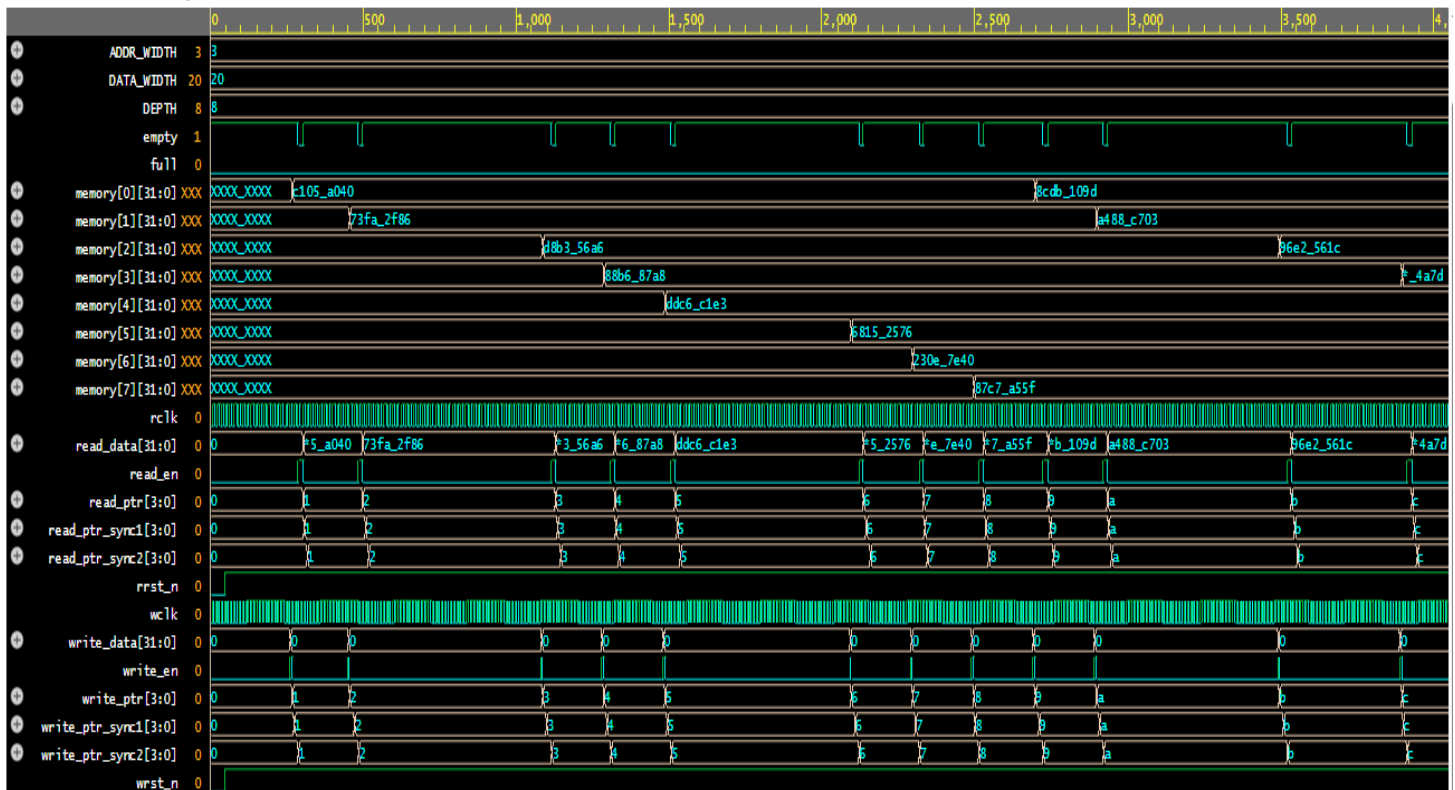
- ACLK
- ARESETn
- AWADDR
- AWVALID
- AWREADY
- WDATA
- WVALID
- WREADY
- BRESP
- BVALID
- ARADDR
- ARVALID
- ARREADY
- RDATA
- RRESP
- RVALID
- RREADY

The waveform demonstrates the AXI4-Lite handshake activity and the corresponding response behavior.

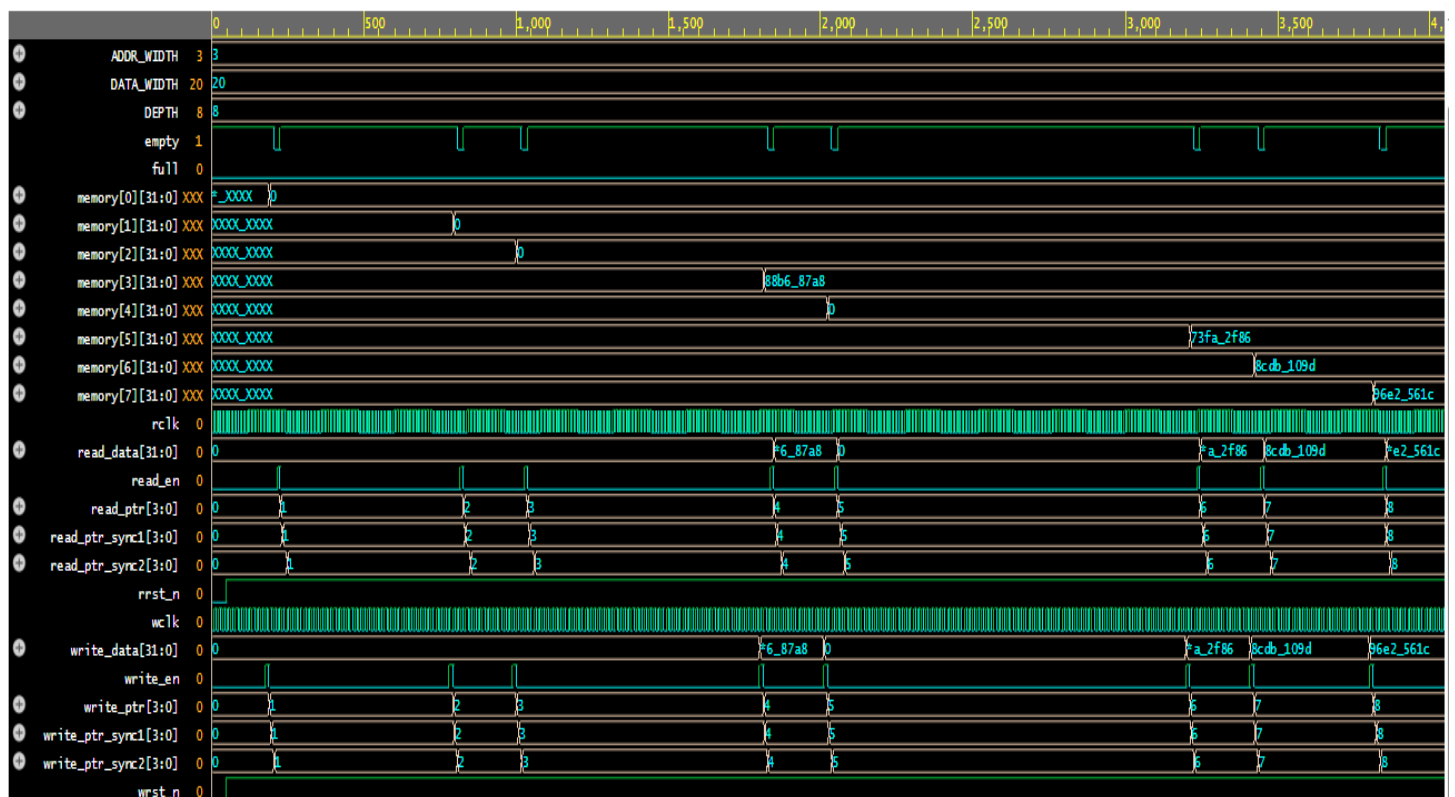
Suggested waveform figures for the final report

AXI TO APB BUS PROTOCOL

- **Figure 11: AXI4-Lite Write Transaction Waveform**

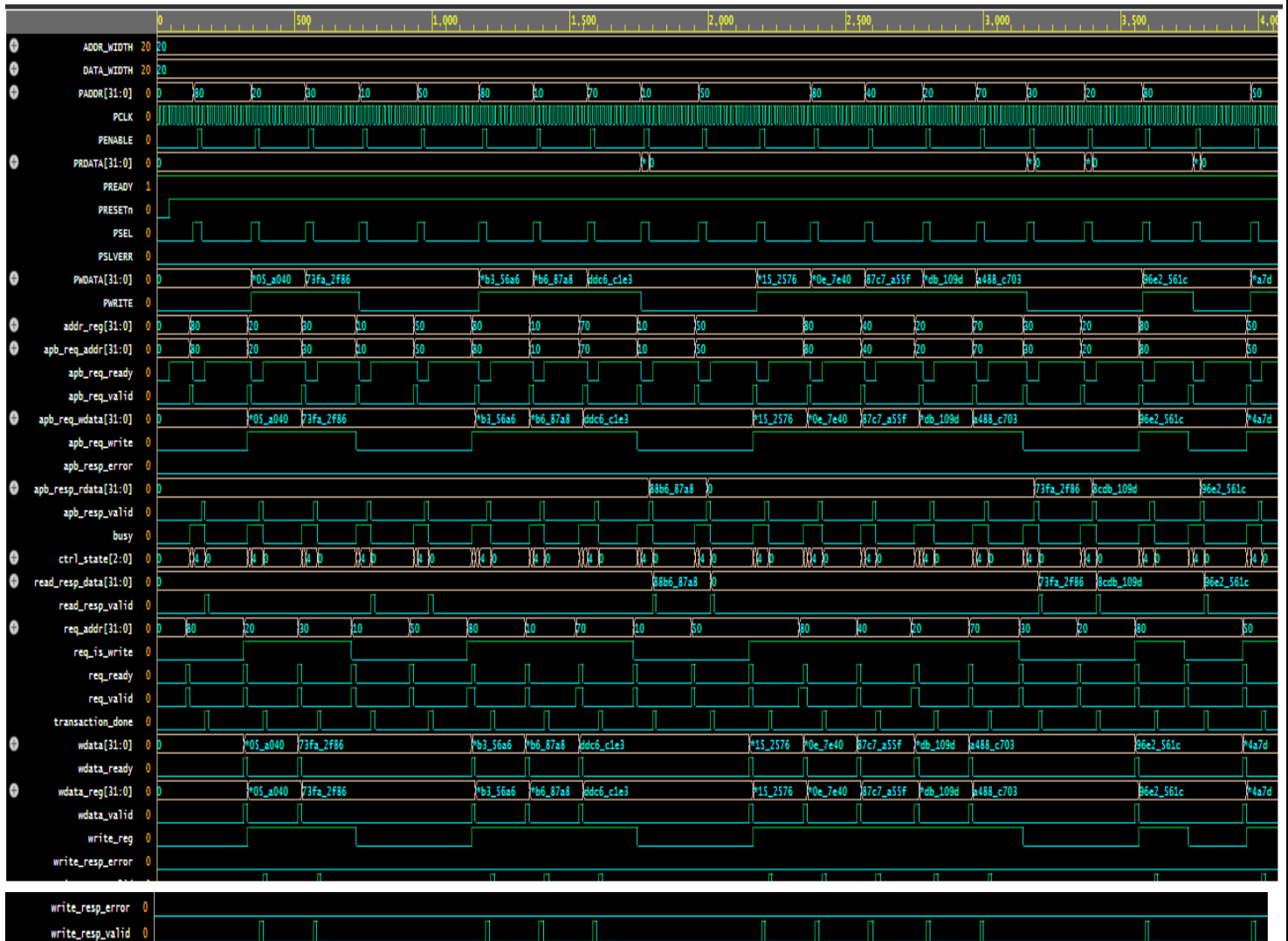


- **Figure 12:** AXI4-Lite Read Transaction Waveform

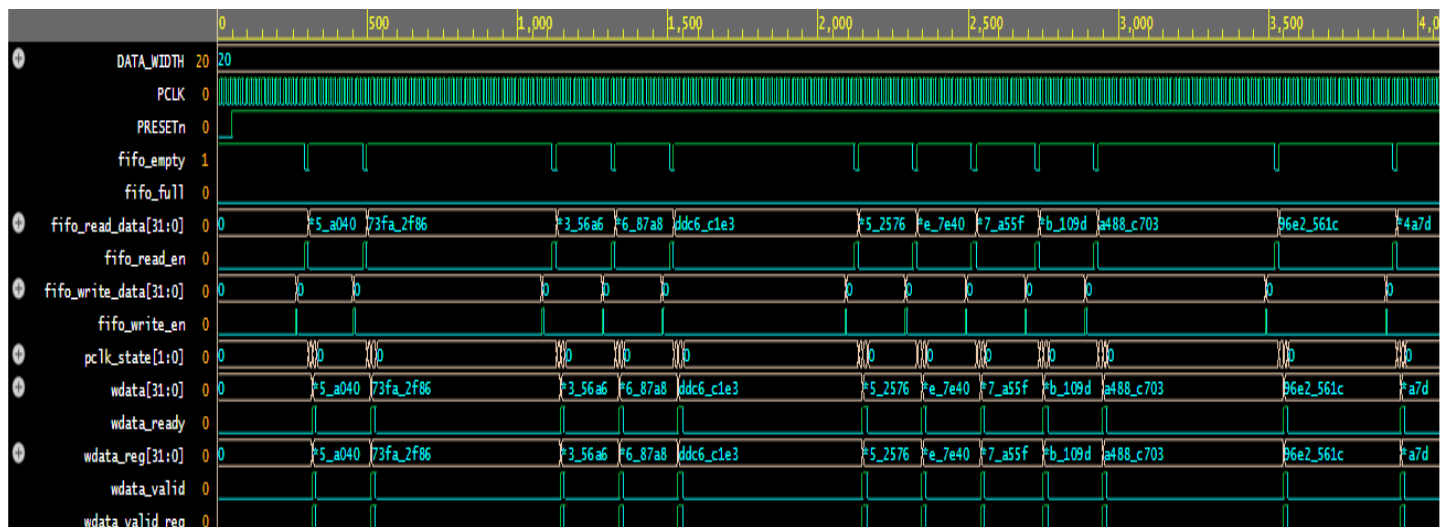


AXI TO APB BUS PROTOCOL

• **Figure 13: Bridge Control Transaction Waveform**

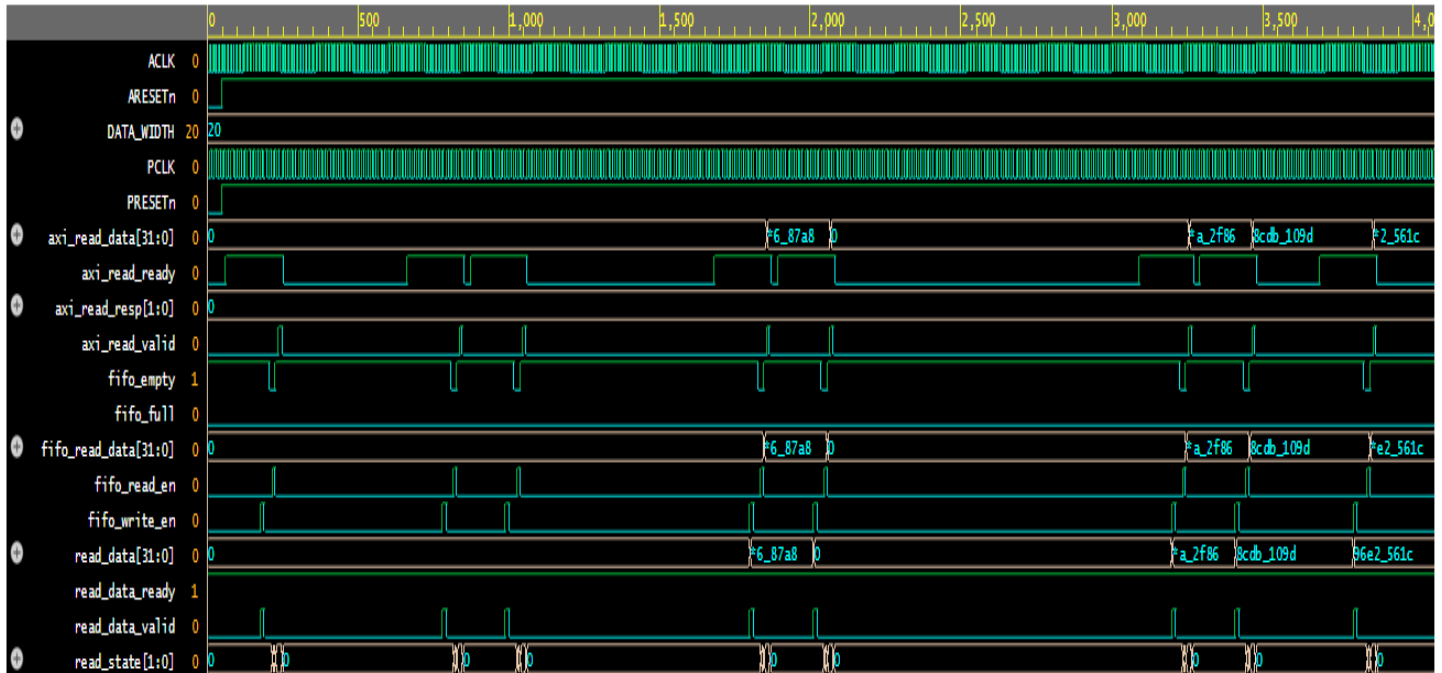


• **Figure 14: APB Write Transfer Waveform**

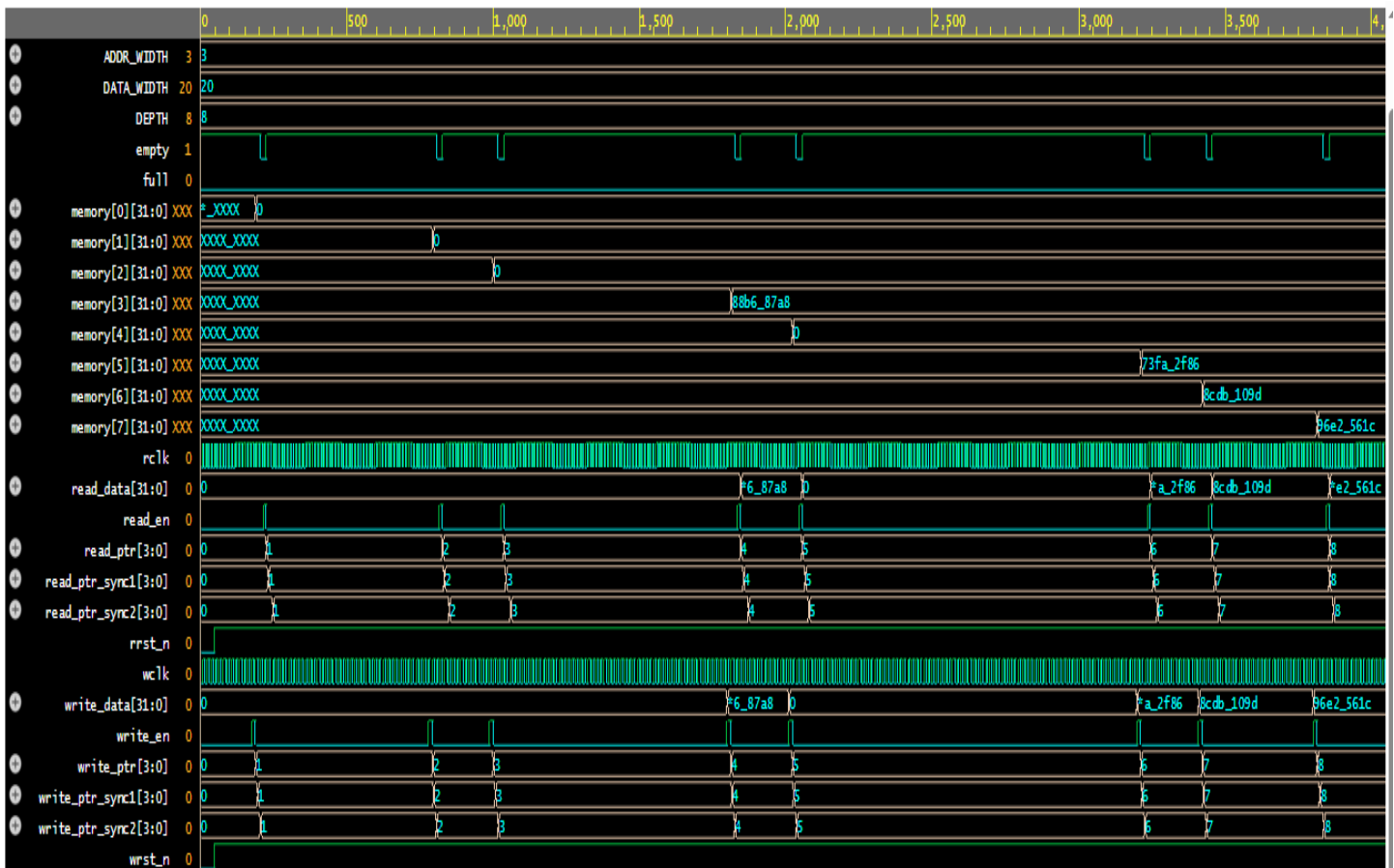


AXI TO APB BUS PROTOCOL

• **Figure 15: APB Read Transfer Waveform**

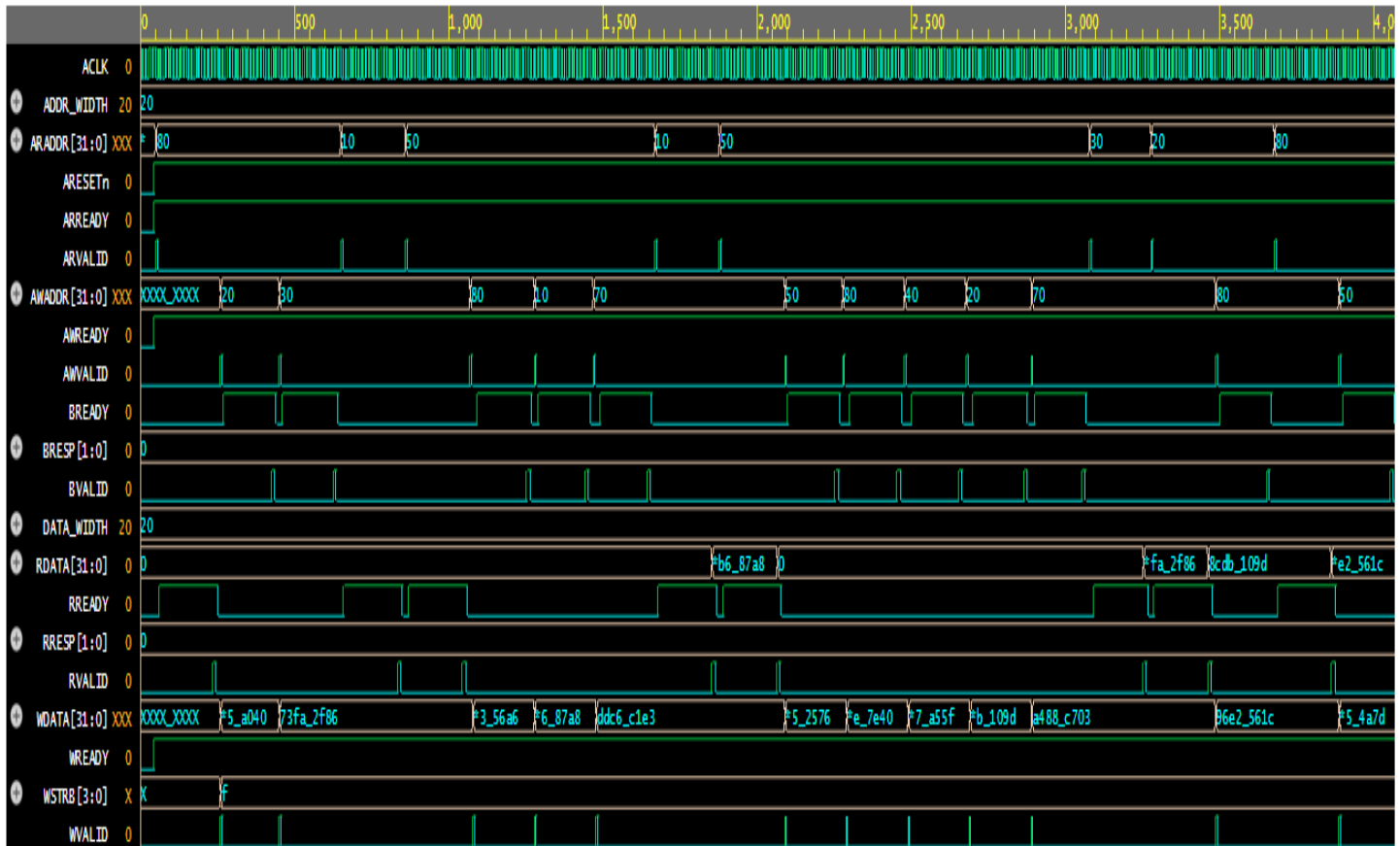


• **Figure 16: Read-after-Write Verification Waveform**



AXI TO APB BUS PROTOCOL

- **Figure 17: Complete AXI-to-APB Transaction Flow**



9.7 Output Log Analysis

The terminal output provides additional evidence of successful verification.

For example:

UVM_INFO axi_uvm_test.sv(115) @ 4070:

[TEST] All 20 AXI transactions completed successfully.

Driver count = 20

The coverage results were:

Operation Coverage = 100.00%

Address Coverage = 87.50%

Operation x Address Coverage = 87.50%

The final UVM report contained:

UVM_INFO : 207

UVM_WARNING : 0

UVM_ERROR : 0

UVM_FATAL : 0

AXI TO APB BUS PROTOCOL

These logs should be included in the report as screenshots from the Xcelium/EDA environment.

```
[2026-09-28 06:49:38 UTC] xrun -Q -unbuffered '-timescale' '1ns/1ns' '-sysv' '-coverage' 'all' '-access' 'rw' -uvmnocdsextra -uvmhome $UVM_HOME $UVM_HOME/src/uvm_macros.svh de
TOOL: xrun 25.03-s001: Started on Sep 28, 2026 at 02:49:39 EDT
xrun: 25.03-s001: (c) Copyright 1995-2025 Cadence Design Systems, Inc.
Top level design units:
  uvm_pkg
  $unit_0x110a026e
  tb_top
xmelab: *W,COVFDP: FSM description matching across instance of a module for "COVFDM" warning has been moved from elaboration to simulation dumping. To restore old matching behav
xmelab: *W,COVFHT: FSM hold transitions (transitions to the current state) are not extracted for any FSM in default mode.
xmelab: *W,COVDCL: By default expression coverage is scored only for Verilog logical operators (|| and &&) and VHDL logical operators (OR, AND, NOR, and NAND), and is scored only
xmelab: *W,COVNOEN: By default, toggle coverage is not supported for systemverilog enumerated nets and variables. To enable toggle coverage of these objects, specify the 'set_to
  logic [DATA_WIDTH-1:0] memory [0:DEPTH-1];
  |
xmelab: *W,COVMD0 (./async_fifo.sv,34|32): Toggle coverage for bit, logic, reg, wire, enum and struct multi-dimensional static arrays and vectors is not supported by default. To
xmelab: *W,CGPIDF: Covergroup "per_instance" option, by default, is set as 0.
xmelab: *N,COVCGN: Coverage configuration file command "set_covergroup -new_instance_reporting" can be specified to improve the scoping and naming of covergroup instances. It may
Loading snapshot worklib.tb_top:sv ..... Done
SVSEED default: 1
xcelium> source /xcelium25.03/tools/xcelium/files/xmsimrc
xcelium> source /xcelium25.03/tools//methodology/UVM/CDNS-1.2/sv/files/tcl/uvm_sim.tcl
xcelium> run
UVM_INFO /xcelium25.03/tools/methodology/UVM/CDNS-1.2/sv/src/base/uvm_root.svh(412) @ 0: reporter [UVM/RELNOTES]
-----
CDNS-UVM-1.2 (25.03-s001)
(C) 2007-2014 Mentor Graphics Corporation
(C) 2007-2014 Cadence Design Systems, Inc.
(C) 2006-2014 Synopsys, Inc.
(C) 2011-2013 Cypress Semiconductor Corp.
(C) 2013-2014 NVIDIA Corporation
-----
```

```
***** IMPORTANT RELEASE NOTES *****

You are using a version of the UVM library that has been compiled
with `UVM_NO_DEPRECATED undefined.
See http://www.eda.org/svdb/view.php?id=3313 for more details.

You are using a version of the UVM library that has been compiled
with `UVM_OBJECT_DO_NOT_NEED_CONSTRUCTOR undefined.
See http://www.eda.org/svdb/view.php?id=3770 for more details.

(Specify +UVM_NO_RELNOTES to turn off this notice)

UVM_INFO @ 0: reporter [RNTST] Running test axi_uvm_test...
UVM_INFO axi_uvm_test.sv(66) @ 0: uvm_test_top [TEST] =====
UVM_INFO axi_uvm_test.sv(72) @ 0: uvm_test_top [TEST] STEP 3.9 FINAL UVM REGRESSION
UVM_INFO axi_uvm_test.sv(78) @ 0: uvm_test_top [TEST] Starting Final AXI UVM Regression
UVM_INFO axi_driver.sv(88) @ 45: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=0 ADDR=00000080 DATA=1bf89705 SEPARATE_AW_W=0
UVM_INFO axi_monitor.sv(313) @ 50: uvm_test_top.env.agent.monitor [AXI_MON] READ ADDRESS: ADDR=00000080
UVM_INFO axi_coverage.sv(109) @ 50: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=0 ADDR=00000080
UVM_INFO axi_scoreboard.sv(104) @ 50: uvm_test_top.env.scoreboard [SCOREBOARD] READ ADDRESS received: ADDR=00000080
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=0 ADDR=00000080 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=0 ADDR=00000080 READY=1
BRIDGE CONTROL: Starting APB READ, ADDR=00000080
APB MEMORY: READ ADDR=00000080 INDEX=32 DATA=00000000
BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0
UVM_INFO axi_monitor.sv(380) @ 235: uvm_test_top.env.agent.monitor [AXI_MON] READ RESPONSE: DATA=00000000 RRESP=00
UVM_INFO axi_scoreboard.sv(151) @ 235: uvm_test_top.env.scoreboard [SCOREBOARD] READ CHECK: ADDR=00000080 has no previous write. DATA=00000000
UVM_INFO axi_driver.sv(319) @ 245: uvm_test_top.env.agent.driver [AXI_DRV] READ completed: ADDR=00000080 DATA=00000000 RRESP=00
```

AXI TO APB BUS PROTOCOL

```
UVM_INFO axi_driver.sv(111) @ 250: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 1
UVM_INFO axi_sequence.sv(77) @ 250: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=0 ADDR=00000080 DATA=1bf89705 SEPARATE_AW_W=0
UVM_INFO axi_driver.sv(88) @ 250: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=1 ADDR=00000020 DATA=c105a040 SEPARATE_AW_W=0
UVM_INFO axi_monitor.sv(116) @ 260: uvm_test_top.env.agent.monitor [AXI_MON] WRITE ADDRESS: ADDR=00000020
UVM_INFO axi_coverage.sv(109) @ 260: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=1 ADDR=00000020
UVM_INFO axi_scoreboard.sv(62) @ 260: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE ADDRESS received: ADDR=00000020
UVM_INFO axi_monitor.sv(182) @ 260: uvm_test_top.env.agent.monitor [AXI_MON] WRITE DATA: DATA=c105a040 WSTRB=f
UVM_INFO axi_scoreboard.sv(78) @ 260: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE CHECK PASS: ADDR=00000020 DATA=c105a040
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=1 ADDR=00000020 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=1 ADDR=00000020 READY=1
BRIDGE CONTROL: Starting APB WRITE, ADDR=00000020 DATA=c105a040
APB MEMORY: WRITE ADDR=00000020 DATA=c105a040
BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0
UVM_INFO axi_monitor.sv(248) @ 425: uvm_test_top.env.agent.monitor [AXI_MON] WRITE RESPONSE: BRESP=00
UVM_INFO axi_driver.sv(244) @ 435: uvm_test_top.env.agent.driver [AXI_DRV] WRITE completed: ADDR=00000020 DATA=c105a040
UVM_INFO axi_driver.sv(111) @ 440: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 2
UVM_INFO axi_sequence.sv(77) @ 440: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=1 ADDR=00000020 DATA=c105a040 SEPARATE_AW_W=0
UVM_INFO axi_driver.sv(88) @ 440: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=1 ADDR=00000030 DATA=73fa2f86 SEPARATE_AW_W=0
UVM_INFO axi_monitor.sv(116) @ 450: uvm_test_top.env.agent.monitor [AXI_MON] WRITE ADDRESS: ADDR=00000030
UVM_INFO axi_coverage.sv(109) @ 450: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=1 ADDR=00000030
UVM_INFO axi_scoreboard.sv(62) @ 450: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE ADDRESS received: ADDR=00000030
UVM_INFO axi_monitor.sv(182) @ 450: uvm_test_top.env.agent.monitor [AXI_MON] WRITE DATA: DATA=73fa2f86 WSTRB=f
UVM_INFO axi_scoreboard.sv(78) @ 450: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE CHECK PASS: ADDR=00000030 DATA=73fa2f86
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=1 ADDR=00000030 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=1 ADDR=00000030 READY=1
BRIDGE CONTROL: Starting APB WRITE, ADDR=00000030 DATA=73fa2f86
APB MEMORY: WRITE ADDR=00000030 DATA=73fa2f86
BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0
UVM_INFO axi_monitor.sv(248) @ 625: uvm_test_top.env.agent.monitor [AXI_MON] WRITE RESPONSE: BRESP=00
UVM_INFO axi_driver.sv(244) @ 635: uvm_test_top.env.agent.driver [AXI_DRV] WRITE completed: ADDR=00000030 DATA=73fa2f86
UVM_INFO axi_driver.sv(111) @ 640: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 3
UVM_INFO axi_sequence.sv(77) @ 640: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=1 ADDR=00000030 DATA=73fa2f86 SEPARATE_AW_W=0
UVM_INFO axi_driver.sv(88) @ 640: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=0 ADDR=00000010 DATA=cbfbc807 SEPARATE_AW_W=0
UVM_INFO axi_monitor.sv(313) @ 650: uvm_test_top.env.agent.monitor [AXI_MON] READ ADDRESS: ADDR=00000010
UVM_INFO axi_coverage.sv(109) @ 650: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=0 ADDR=00000010
UVM_INFO axi_scoreboard.sv(104) @ 650: uvm_test_top.env.scoreboard [SCOREBOARD] READ ADDRESS received: ADDR=00000010
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=0 ADDR=00000010 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=0 ADDR=00000010 READY=1
BRIDGE CONTROL: Starting APB READ, ADDR=00000010
APB MEMORY: READ ADDR=00000010 INDEX=4 DATA=00000000
BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0
UVM_INFO axi_monitor.sv(380) @ 835: uvm_test_top.env.agent.monitor [AXI_MON] READ RESPONSE: DATA=00000000 RRESP=00
UVM_INFO axi_scoreboard.sv(151) @ 835: uvm_test_top.env.scoreboard [SCOREBOARD] READ CHECK: ADDR=00000010 has no previous write. DATA=00000000
UVM_INFO axi_driver.sv(319) @ 845: uvm_test_top.env.agent.driver [AXI_DRV] READ completed: ADDR=00000010 DATA=00000000 RRESP=00
UVM_INFO axi_driver.sv(111) @ 850: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 4
UVM_INFO axi_sequence.sv(77) @ 850: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=0 ADDR=00000010 DATA=cbfbc807 SEPARATE_AW_W=0
UVM_INFO axi_driver.sv(88) @ 850: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=0 ADDR=00000050 DATA=c90a69c2 SEPARATE_AW_W=1
UVM_INFO axi_monitor.sv(313) @ 860: uvm_test_top.env.agent.monitor [AXI_MON] READ ADDRESS: ADDR=00000050
UVM_INFO axi_coverage.sv(109) @ 860: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=0 ADDR=00000050
UVM_INFO axi_scoreboard.sv(104) @ 860: uvm_test_top.env.scoreboard [SCOREBOARD] READ ADDRESS received: ADDR=00000050
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=0 ADDR=00000050 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=0 ADDR=00000050 READY=1
BRIDGE CONTROL: Starting APB READ, ADDR=00000050
```

AXI TO APB BUS PROTOCOL

```
APB MEMORY: READ ADDR=00000050 INDEX=20 DATA=00000000
BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0
UVM_INFO axi_monitor.sv(380) @ 1045: uvm_test_top.env.agent.monitor [AXI_MON] READ RESPONSE: DATA=00000000 RRESP=00
UVM_INFO axi_scoreboard.sv(151) @ 1045: uvm_test_top.env.scoreboard [SCOREBOARD] READ CHECK: ADDR=00000050 has no previous write. DATA=00000000
UVM_INFO axi_driver.sv(319) @ 1055: uvm_test_top.env.agent.driver [AXI_DRV] READ completed: ADDR=00000050 DATA=00000000 RRESP=00
UVM_INFO axi_driver.sv(111) @ 1060: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 5
UVM_INFO axi_sequence.sv(77) @ 1060: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=0 ADDR=00000050 DATA=c90a69c2 SEPARATE_AW_W=1
UVM_INFO axi_driver.sv(88) @ 1060: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=1 ADDR=00000080 DATA=d8b356a6 SEPARATE_AW_W=1
UVM_INFO axi_monitor.sv(116) @ 1070: uvm_test_top.env.agent.monitor [AXI_MON] WRITE ADDRESS: ADDR=00000080
UVM_INFO axi_coverage.sv(109) @ 1070: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=1 ADDR=00000080
UVM_INFO axi_scoreboard.sv(62) @ 1070: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE ADDRESS received: ADDR=00000080
UVM_INFO axi_monitor.sv(182) @ 1080: uvm_test_top.env.agent.monitor [AXI_MON] WRITE DATA: DATA=d8b356a6 WSTRB=f
UVM_INFO axi_scoreboard.sv(78) @ 1080: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE CHECK PASS: ADDR=00000080 DATA=d8b356a6
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=1 ADDR=00000080 READY=0
REQUEST BRIDGE: REQ VALID=1 WRITE=1 ADDR=00000080 READY=0
BRIDGE CONTROL: Write data received = d8b356a6
BRIDGE CONTROL: Request received. WRITE=1 ADDR=00000080 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=1 ADDR=00000080 READY=1
BRIDGE CONTROL: Starting APB WRITE, ADDR=00000080 DATA=d8b356a6
APB MEMORY: WRITE ADDR=00000080 DATA=d8b356a6
BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0
UVM_INFO axi_monitor.sv(248) @ 1255: uvm_test_top.env.agent.monitor [AXI_MON] WRITE RESPONSE: BRESP=00
UVM_INFO axi_driver.sv(244) @ 1265: uvm_test_top.env.agent.driver [AXI_DRV] WRITE completed: ADDR=00000080 DATA=d8b356a6
UVM_INFO axi_driver.sv(111) @ 1270: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 6
UVM_INFO axi_sequence.sv(77) @ 1270: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=1 ADDR=00000080 DATA=d8b356a6 SEPARATE_AW_W=1
UVM_INFO axi_driver.sv(88) @ 1270: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=1 ADDR=00000010 DATA=88b687a8 SEPARATE_AW_W=0
UVM_INFO axi_monitor.sv(116) @ 1280: uvm_test_top.env.agent.monitor [AXI_MON] WRITE ADDRESS: ADDR=00000010
UVM_INFO axi_coverage.sv(109) @ 1280: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=1 ADDR=00000010
UVM_INFO axi_scoreboard.sv(62) @ 1280: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE ADDRESS received: ADDR=00000010
UVM_INFO axi_monitor.sv(182) @ 1280: uvm_test_top.env.agent.monitor [AXI_MON] WRITE DATA: DATA=88b687a8 WSTRB=f
UVM_INFO axi_scoreboard.sv(78) @ 1280: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE CHECK PASS: ADDR=00000010 DATA=88b687a8
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=1 ADDR=00000010 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=1 ADDR=00000010 READY=1
BRIDGE CONTROL: Starting APB WRITE, ADDR=00000010 DATA=88b687a8
APB MEMORY: WRITE ADDR=00000010 DATA=88b687a8
BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0
UVM_INFO axi_monitor.sv(248) @ 1445: uvm_test_top.env.agent.monitor [AXI_MON] WRITE RESPONSE: BRESP=00
UVM_INFO axi_driver.sv(244) @ 1455: uvm_test_top.env.agent.driver [AXI_DRV] WRITE completed: ADDR=00000010 DATA=88b687a8
UVM_INFO axi_driver.sv(111) @ 1460: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 7
UVM_INFO axi_sequence.sv(77) @ 1460: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=1 ADDR=00000010 DATA=88b687a8 SEPARATE_AW_W=0
UVM_INFO axi_driver.sv(88) @ 1460: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=1 ADDR=00000070 DATA=ddc6c1e3 SEPARATE_AW_W=1
UVM_INFO axi_monitor.sv(116) @ 1470: uvm_test_top.env.agent.monitor [AXI_MON] WRITE ADDRESS: ADDR=00000070
UVM_INFO axi_coverage.sv(109) @ 1470: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=1 ADDR=00000070
UVM_INFO axi_scoreboard.sv(62) @ 1470: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE ADDRESS received: ADDR=00000070
UVM_INFO axi_monitor.sv(182) @ 1480: uvm_test_top.env.agent.monitor [AXI_MON] WRITE DATA: DATA=ddc6c1e3 WSTRB=f
UVM_INFO axi_scoreboard.sv(78) @ 1480: uvm_test_top.env.scoreboard [SCOREBOARD] WRITE CHECK PASS: ADDR=00000070 DATA=ddc6c1e3
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=1 ADDR=00000070 READY=0
REQUEST BRIDGE: REQ VALID=1 WRITE=1 ADDR=00000070 READY=0
BRIDGE CONTROL: Write data received = ddc6c1e3
BRIDGE CONTROL: Request received. WRITE=1 ADDR=00000070 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=1 ADDR=00000070 READY=1
BRIDGE CONTROL: Starting APB WRITE, ADDR=00000070 DATA=ddc6c1e3
APB MEMORY: WRITE ADDR=00000070 DATA=ddc6c1e3
BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0
UVM_INFO axi_monitor.sv(248) @ 1645: uvm_test_top.env.agent.monitor [AXI_MON] WRITE RESPONSE: BRESP=00
UVM_INFO axi_driver.sv(244) @ 1655: uvm_test_top.env.agent.driver [AXI_DRV] WRITE completed: ADDR=00000070 DATA=ddc6c1e3
```

AXI TO APB BUS PROTOCOL

```
UVM_INFO axi_driver.sv(111) @ 1660: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 8
UVM_INFO axi_sequence.sv(77) @ 1660: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=1 ADDR=00000070 DATA=ddc6c1e3 SEPARATE_AW_W=1
UVM_INFO axi_driver.sv(88) @ 1660: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=0 ADDR=00000010 DATA=40be822c SEPARATE_AW_W=1
UVM_INFO axi_monitor.sv(313) @ 1670: uvm_test_top.env.agent.monitor [AXI_MON] READ ADDRESS: ADDR=00000010
UVM_INFO axi_coverage.sv(109) @ 1670: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=0 ADDR=00000010
UVM_INFO axi_scoreboard.sv(104) @ 1670: uvm_test_top.env.scoreboard [SCOREBOARD] READ ADDRESS received: ADDR=00000010
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=0 ADDR=00000010 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=0 ADDR=00000010 READY=1
BRIDGE CONTROL: Starting APB READ, ADDR=00000010
APB MEMORY: READ ADDR=00000010 INDEX=4 DATA=88b687a8
BRIDGE CONTROL: APB RESPONSE received. RDATA=88b687a8 ERROR=0
UVM_INFO axi_monitor.sv(380) @ 1855: uvm_test_top.env.agent.monitor [AXI_MON] READ RESPONSE: DATA=88b687a8 RRESP=00
UVM_INFO axi_scoreboard.sv(123) @ 1855: uvm_test_top.env.scoreboard [SCOREBOARD] READ CHECK PASS: ADDR=00000010 EXPECTED=88b687a8 ACTUAL=88b687a8
UVM_INFO axi_driver.sv(319) @ 1865: uvm_test_top.env.agent.driver [AXI_DRV] READ completed: ADDR=00000010 DATA=88b687a8 RRESP=00
UVM_INFO axi_driver.sv(111) @ 1870: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 9
UVM_INFO axi_sequence.sv(77) @ 1870: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=0 ADDR=00000010 DATA=40be822c SEPARATE_AW_W=1
UVM_INFO axi_driver.sv(88) @ 1870: uvm_test_top.env.agent.driver [AXI_DRV] Driving transaction: WRITE=0 ADDR=00000050 DATA=4d7610cb SEPARATE_AW_W=0
UVM_INFO axi_monitor.sv(313) @ 1880: uvm_test_top.env.agent.monitor [AXI_MON] READ ADDRESS: ADDR=00000050
UVM_INFO axi_coverage.sv(109) @ 1880: uvm_test_top.env.coverage [COVERAGE] Coverage sampled: WRITE=0 ADDR=00000050
UVM_INFO axi_scoreboard.sv(104) @ 1880: uvm_test_top.env.scoreboard [SCOREBOARD] READ ADDRESS received: ADDR=00000050
REQUEST BRIDGE: FIFO READ, EMPTY=0
BRIDGE CONTROL: Request received. WRITE=0 ADDR=00000050 READY=1
REQUEST BRIDGE: REQ VALID=1 WRITE=0 ADDR=00000050 READY=1
BRIDGE CONTROL: Starting APB READ, ADDR=00000050
APB MEMORY: READ ADDR=00000050 INDEX=20 DATA=00000000
BRIDGE CONTROL: APB RESPONSE received. RDATA=00000000 ERROR=0
UVM_INFO axi_monitor.sv(380) @ 2065: uvm_test_top.env.agent.monitor [AXI_MON] READ RESPONSE: DATA=00000000 RRESP=00
UVM_INFO axi_scoreboard.sv(151) @ 2065: uvm_test_top.env.scoreboard [SCOREBOARD] READ CHECK: ADDR=00000050 has no previous write. DATA=00000000
UVM_INFO axi_driver.sv(319) @ 2075: uvm_test_top.env.agent.driver [AXI_DRV] READ completed: ADDR=00000050 DATA=00000000 RRESP=00

UVM_INFO axi_monitor.sv(248) @ 4055: uvm_test_top.env.agent.monitor [AXI_MON] WRITE RESPONSE: BRESP=00
UVM_INFO axi_driver.sv(244) @ 4065: uvm_test_top.env.agent.driver [AXI_DRV] WRITE completed: ADDR=00000050 DATA=60b54a7d
UVM_INFO axi_driver.sv(111) @ 4070: uvm_test_top.env.agent.driver [AXI_DRV] Transaction completed. Total completed = 20
UVM_INFO axi_sequence.sv(77) @ 4070: uvm_test_top.env.agent.sequencer@seq [AXI_SEQ] Generated transaction: WRITE=1 ADDR=00000050 DATA=60b54a7d SEPARATE_AW_W=0
UVM_INFO axi_uvm_test.sv(103) @ 4070: uvm_test_top [TEST] Sequence completed. Checking driver transaction count...
UVM_INFO axi_uvm_test.sv(115) @ 4070: uvm_test_top [TEST] All 20 AXI transactions completed successfully. Driver count = 20
UVM_INFO axi_uvm_test.sv(135) @ 4070: uvm_test_top [TEST] Final AXI UVM Regression Completed
UVM_INFO /xcelium25.03/tools/methodology/UVM/CDNS-1.2/sv/src/base/uvm_objection.svh(1271) @ 4070: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
UVM_INFO axi_coverage.sv(122) @ 4070: uvm_test_top.env.coverage [COVERAGE] =====
UVM_INFO axi_coverage.sv(128) @ 4070: uvm_test_top.env.coverage [COVERAGE] UVM FUNCTIONAL COVERAGE RESULTS
UVM_INFO axi_coverage.sv(137) @ 4070: uvm_test_top.env.coverage [COVERAGE] Operation Coverage = 100.00%
UVM_INFO axi_coverage.sv(146) @ 4070: uvm_test_top.env.coverage [COVERAGE] Address Coverage = 87.50%
UVM_INFO axi_coverage.sv(155) @ 4070: uvm_test_top.env.coverage [COVERAGE] Operation x Address Coverage = 87.50%
UVM_INFO axi_coverage.sv(161) @ 4070: uvm_test_top.env.coverage [COVERAGE] =====
UVM_INFO /xcelium25.03/tools/methodology/UVM/CDNS-1.2/sv/src/base/uvm_report_server.svh(851) @ 4070: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 207
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
```

AXI TO APB BUS PROTOCOL

```
[AXI_DRV] 60
[AXI_MON] 52
[AXI_SEQ] 20
[COVERAGE] 26
[RNTST] 1
[SCOREBOARD] 40
[TEST] 6
[TEST_DONE] 1
[UVM/RELNOTES] 1
```

Simulation complete via \$finish(1) at time 4070 NS + 64

```
/xcelium25.03/tools/methodology/UVM/CDNS-1.2/sv/src/base/uvm_root.svh:543 $finish;
```

```
xcelium> exit
```

```
xmsim: *N,COVCGN: Coverage configuration file command "set_covergroup -new_instance_reporting" can be specified to improve the scoping and naming of covergroup instances. It
xmsim: *W,CGPIZE: Instance coverage for covergroup instance "axi_operation_cg" will not be dumped to database as per_instance option value is set to 0:./axi_coverage.sv, 13.
xmsim: *W,CGPIZE: Instance coverage for covergroup instance "axi_address_cg" will not be dumped to database as per_instance option value is set to 0:./axi_coverage.sv, 26.
xmsim: *W,CGPIZE: Instance coverage for covergroup instance "axi_cross_cg" will not be dumped to database as per_instance option value is set to 0:./axi_coverage.sv, 45.
xmsim: *W,CGPIZE: Instance coverage for covergroup instance "axi_operation_cg" will not be dumped to database as per_instance option value is set to 0:./axi_coverage.sv, 13.
xmsim: *W,CGPIZE: Instance coverage for covergroup instance "axi_address_cg" will not be dumped to database as per_instance option value is set to 0:./axi_coverage.sv, 26.
xmsim: *W,CGPIZE: Instance coverage for covergroup instance "axi_cross_cg" will not be dumped to database as per_instance option value is set to 0:./axi_coverage.sv, 45.
```

coverage setup:

```
workdir : ./cov_work
dutinst : tb_top(tb_top)
scope : scope
testname : test
```

coverage files:

```
model(design data) : ./cov_work/scope/icc_1b6582e2_0e9290ce.ucm
data : ./cov_work/scope/test/icc_1b6582e2_0e9290ce.ucd
```

TOOL: xrun 25.03-s001: Exiting on Sep 28, 2026 at 02:49:49 EDT (total: 00:00:10)

Finding VCD file...

```
./step3_6_uvm.vcd
```

```
[2026-09-28 06:49:49 UTC] Opening EPWave...
```

```
Done
```

CHAPTER 10

CONCLUSION AND FUTURE WORK:

10.1 Conclusion:

The project successfully implemented and verified an AXI4-Lite to APB bus protocol bridge using SystemVerilog RTL and UVM-based verification.

The bridge successfully handled AXI4-Lite read and write transactions and converted them into corresponding APB operations. The APB memory model responded correctly to the generated transactions, while the UVM scoreboard verified the returned data.

The final regression completed 20 out of 20 AXI transactions successfully, with zero UVM errors, zero warnings, and zero fatal errors. Functional coverage reached 100% for operation coverage, 87.5% for address coverage, and 87.5% for operation × address coverage.

The successful read-after-write checks further demonstrate that data written through the bridge could later be retrieved correctly through an AXI4-Lite read.

Overall, the project provided practical experience in RTL design, AMBA protocols, FIFO-based communication, SystemVerilog, UVM, functional coverage, scoreboarding, and waveform-based debugging.

10.2 Limitations:

Although the project successfully demonstrates the basic AXI4-Lite to APB conversion process, several limitations remain. The current implementation does not target:

- Full AXI4 burst transactions.
- AXI transaction IDs.
- Out-of-order transactions.
- Multiple outstanding AXI transactions.
- Multiple APB slave selection.
- Advanced APB peripheral behaviour.
- Comprehensive error injection.
- Complete protocol compliance coverage.
- Full synthesis, timing, and power analysis.

The current functional coverage also contains unhit address and operation/address combinations, as reflected by the 87.5% address and cross coverage.

10.3 Future Work:

The project can be extended in several directions.

1. Full AXI Support: The bridge can be extended from AXI4-Lite to support additional AXI4 features such as burst transactions and transaction IDs.

2. Multiple APB Slaves: A peripheral decoder can be added to allow the AXI master to access multiple APB slaves.

3. Improved Error Handling: Additional APB error scenarios can be introduced and mapped to appropriate AXI responses.

4. Enhanced UVM Verification: The verification environment can be expanded with additional sequences, protocol assertions, negative testing, and constrained-random scenarios.

5. Improved Functional Coverage: Additional address bins and operation/address combinations can be generated to achieve higher coverage.

6. Assertion-Based Verification: A larger set of SystemVerilog Assertions can be implemented to automatically detect protocol violations.

7. Synthesis and Timing Analysis:

The RTL can be synthesized to evaluate:

- Area.
- Timing.
- Resource utilization.
- Maximum operating frequency.

8. FPGA Implementation: The bridge can be synthesized and tested on an FPGA development board.

9. Power Analysis: Power estimation can be performed to evaluate the efficiency of the design.

10. Reusable Verification IP: The UVM environment can be further developed into a reusable verification component for AXI4-Lite/APB bridge designs.

REFERENCES

- [1] Arm Limited, “AMBA AXI and ACE Protocol Specification: AXI3, AXI4, and AXI4-Lite, ACE and ACE-Lite,” ARM IHI 0022H, 2020. [Online]. Available: <https://developer.arm.com/>
- [2] Arm Limited, “AMBA APB Protocol Specification,” AMBA Specification, Arm Limited. [Online]. Available: <https://www.arm.com/architecture/system-architectures/amba/amba-specifications>
- [3] Accellera Systems Initiative, “Universal Verification Methodology (UVM) 1.2 User’s Guide,” Oct. 2015. [Online]. Available: <https://www.accellera.org/>
- [4] Accellera Systems Initiative, “Universal Verification Methodology (UVM) 1.2 Class Reference,” Accellera Systems Initiative, 2014. [Online]. Available: <https://www.accellera.org/>
- [5] IEEE, “IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language,” IEEE Std 1800-2017, 2018.
- [6] IEEE, “IEEE Standard for Universal Verification Methodology Language—IEEE Std 1800.2-2020,” IEEE, 2020.
- [7] ISO/IEC/IEEE, “Systems and Software Engineering—Life Cycle Processes—Requirements Engineering,” ISO/IEC/IEEE 29148:2018, Nov. 2018. [Online]. Available: <https://standards.ieee.org/ieee/29148/6937/>
- [8] Cadence Design Systems, “Xcelium Parallel Simulator,” Cadence Design Systems. [Online]. Available: <https://www.cadence.com/>
- [9] Cadence Design Systems, “Xcelium User Documentation,” Cadence Design Systems.
- [10] Siemens EDA, “SystemVerilog and Universal Verification Methodology Resources,” Siemens Digital Industries Software. [Online]. Available: <https://eda.sw.siemens.com/>
- [11] IEEE, “IEEE Standard for Software and Systems Test Documentation,” IEEE Std 29119, IEEE.
- [12] Accellera Systems Initiative, “Universal Verification Methodology (UVM) Working Group,” Accellera Systems Initiative. [Online]. Available: <https://www.accellera.org/activities/working-groups/uvm>

APPENDIX A

FINAL VERIFICATION COMMAND

The final simulation was executed using Xcelium with a command similar to:

```
xrun -Q -unbuffered -timescale 1ns/1ns -sysv -coverage all \  
-access +rw -uvmnocdnsextra -uvmhome $UVM_HOME \  
$UVM_HOME/src/uvm_macros.svh design.sv testbench.sv
```

APPENDIX B

IMPORTANT FINAL LOG

The final regression reported:

All 20 AXI transactions completed successfully.

Driver count = 20

Coverage:

Operation Coverage = 100.00%

Address Coverage = 87.50%

Operation x Address Coverage = 87.50%

UVM report:

UVM_INFO : 207

UVM_WARNING 0

UVM_ERROR : 0

UVM_FATAL : 0

Simulation completion:

Simulation complete via \$finish

at time 4070 NS

FINAL PROJECT SUMMARY

The completed project demonstrates the design and verification of a functional AXI4-Lite to APB bridge. The bridge receives AXI4-Lite transactions, processes them through the bridge control and FIFO-based communication logic, generates APB transfers, communicates with an APB memory model, and returns the resulting response to the AXI4-Lite side.

The UVM environment successfully generated and monitored transactions, checked data using a scoreboard, and measured functional coverage. The final regression achieved 20/20 successful transactions with zero UVM errors, while achieving 100% operation coverage, 87.5% address coverage, and 87.5% operation $\times$ address coverage.

The project therefore provides a complete learning and implementation flow covering protocol understanding $\rightarrow$ RTL design $\rightarrow$ bridge implementation $\rightarrow$ UVM verification $\rightarrow$ functional coverage $\rightarrow$ waveform analysis $\rightarrow$ final evaluation.